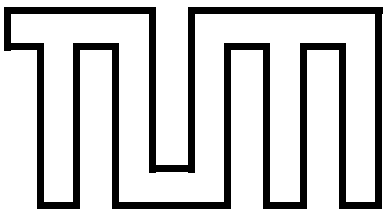



**Technische Universität München**
**Fakultät für Informatik**


# Diplomarbeit in Informatik

Sichere asynchrone Aktualisierungsbenachrichtigungen für ein verteiltes Dateisystem

Secure Asynchronous Change Notifications for a Distributed File System


Cand.-Inf. Bernhard Amann


| | |
|---|---|
| Aufgabensteller: | Prof. Dr. Thomas Fuhrmann<br>Lehrstuhl für Netzwerkarchitekturen, Technische Universität München |
| Betreuer: | Dipl.-Ing. Kendy Kutzner<br>Lehrstuhl Systemarchitektur, Universität Karlsruhe (TH) |
| Beginn: | 15.5.2007 |
| Abgabe: | 15.11.2007 |

# Erklärung

Ich versichere, dass ich diese Diplomarbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Berg, den 15. November 2007 _____
Ort, Datum Unterschrift

# Abstract

Distributed file systems have been a topic of interest for a long time [35] and there are many file systems that are distributed in one way or another. However most distributed file systems are only reasonably usable within a local network of computers and some main tasks are still delegated to a very small number of servers.

Today with the advent of Peer-to-Peer technology [12], distributed file systems that work on top of Peer-to-Peer systems can be built. These systems can be built with no or much less centralised components and are usable on a global scale.

The System Architecture Group at the University of Karlsruhe in Germany has developed such a file system [33], which is built on top of a structured overlay network and uses Distributed Hash Tables [62] to store and access the information.

One problem with this approach is, that each file system can only be accessed with the help of an identifier, which changes whenever a file system is modified. All clients have to be notified of the new identifier in a secure, fast and reliable way.

Usually the strategy to solve this type of problem is an encrypted multicast. This thesis presents and analyses several strategies of using multicast distributions to solve this problem and then unveils our final solution based on the Subset Difference method proposed by Naor et al. in [42].

# Zusammenfassung

Die Informatik beschäftigt sich schon seit längerer Zeit [35] mit verteilten Dateisystemen. Bis heute sind die meisten dieser Systeme aber leider nur in lokalen Netzen vernünftig benutzbar und manche der Hauptaufgaben werden immer noch nur von einer kleinen Nummer zentraler Server übernommen.

Heutzutage kann man aber mit der Hilfe von Peer-to-Peer Netztechnologien [12] verteilte Dateisysteme erstellen, welche ohne oder nur mit sehr wenigen zentralen Komponenten auskommen und weltweit nutzbar sind.

Die Systemarchitekturgruppe der Universität Karlsruhe (TH) hat ein solches verteiltes Dateisystem erstellt [33], welches auf einem strukturierten Peer-to-Peer Netz aufbaut. Verteilte Hashtabellen (Distributed Hash Tables; DHTs) [62] werden verwendet um die Daten zu speichern und wieder abzurufen.

Ein Problem dieses Ansatzes ist, dass man auf jedes Dateisystem nur dann zugreifen kann, wenn man einen Schlüssel kennt, der jedesmal wechselt wenn sich der Inhalt des Dateisystems ändert. Jeder der auf das Dateisystem zugreift, muss nach einer Änderung über den neuen Schlüssel informiert werden; und dies muss sicher, schnell und zuverlässig geschehen.

Normalerweise benützt man bei Problemen dieser Art einen verschlüsselten Multicast. Diese Arbeit präsentiert und analysiert erst Strategien, wie wir unser Problem mit einem verschlüsselten Multicast lösen könnten. Schlussendlich stellen wir aber unseren neuen Ansatz vor, der auf der “Subset Diffence” Methode die von Naor und anderen vorgeschlagen wurde [42] beruht.

# Contents

# 1. Introduction

In the last years, Peer-to-Peer systems have become much more important in our day-to-day Internet use. Today the use of Peer-to-Peer systems is no longer restricted to the simple file-sharing approach of the Napster [see 12, p. 36] generation. Many applications that are in common use today employ Peer-to-Peer protocols. Examples include the Internet telephony software Skype [54], the Steam gaming platform [55] or Internet television/video systems like Joost [26], Zattoo [69], or the BBC iPlayer [2] which has also attracted the attention of mainstream media [e.g. 65].

But there are other interesting applications for Peer-to-Peer technology that have not been exhaustively tested until today. One application that could profit from Peer-to-Peer technologies is a file system. With the use of these technologies, the data can be spread across many computers.

This is particularly interesting in industries like the pharmaceutical industry, where data files are used that in many cases have got a size of several hundred megabytes. Today sites often have to download plenty of these files to be able to run their simulations, even when the applications only need a fraction of each file to run [33].

As a solution to this problem, the IGOR File System (IgorFs) [33] has been developed by the research group of Thomas Fuhrmann at the Technical University of Karlsruhe.

The IGOR File System has been built on top of the IGOR Peer-to-Peer system, an overlay network that combines Peer-to-Peer technology and service orientation. It can deliver messages in $O(\log N)$ hops while maintaining connections to $O(\log N)$ other nodes ($N$ being the total number of nodes in the network) [33].

The IGOR File System is especially suited for distrubuted institutions that share large data files between their nodes, but where every node only reads a relatively small portition of the whole file.

At the outset of this thesis, IgorFs was able to share an unmodified file system between members of the Peer-to-Peer network by giving them the hash- and encryption keys of the file system's supernode. But the nodes could not track modifications of the original file system – they only had a static view of the state the file system had, at the moment the supernode was written.

The aim of this work was to devise a method that allows us to securely notify authorised clients that a new file system revision is available.

In this paper, several possibilities for archieving this goal are examined and the final implementation is shown in detail.

In chapter two, we will take a look at the technical background of our problem. We will give an overview of Peer-to-Peer systems in general and IGOR in specific. Moreover we will also study IgorFs.

In chapter three, we will examine the common approaches for Publish/Subscribe systems and show a possible solution, that is based on the idea of an encrypted multicast.

In chapter four, we introduce the Subset Difference method for encryption. We first show how this encryption algorithm could be used to establish a multicast overlay network with very desirable properties and then present our final problem solution in the form of a push/pull method with the possibility to extend it to a soft-state multicast.

In chapter five, we present out cryptographic implementation of the Subset Difference method. We show the entire cryptographic process from key generation via user revocation to the encryption and decryption of messages.

In chapter six, we give a short outline of the changes and additions that were made to IgorFs. We describe the internal setup of IgorFs and how the cryptographic algorithms are used to insert and access file system snapshots.

Chapter seven shows the way in which we tested the new components of IgorFs for functionality.

Finally chapter 8 concludes this thesis. A short summary of our work and it's difference to other approaches as well as an outlook to possible future extensions is given.

# 2. Background

## 2.1. Peer-to-Peer Networks

Classical client-server systems (like an HTTP Server) are offering a service which other systems (the clients) can access. This method is very simple, the only information a client needs is the address of the server. The problems with this approach are also obvious: the server is a single-point of failure and can easily be overwhelmed with requests, for example if there is a spike of popularity for a hosted site[1].

Peer-to-Peer (P2P) networks appeared around the year 2000 when broadband Internet became more widely available [12, p.35]. Today there are several different types of P2P networks, the difference of which we will show later. P2P networks can remove several of the restrictions that are present in the client-server model.

In P2P networks each node of the network acts as both, client and server. Information is forwarded by each node and usually there is no central entity as well as no single point of failure exist. The P2P networks grow with the number of nodes that are present. If there are many nodes and as a consequence of that, a high volume of traffic or queries, there are also more nodes available being able to fill the requests. In an ideal P2P network there never should be a bottleneck, once again because there are no central components.

P2P networks can be subdivided into two big types: structured and unstructured ones. The first evolutional step of P2P networks were the unstructured P2P networks.

### 2.1.1. Centralised Peer-to-Peer Networks

A prominent example for an unstructured P2P network is Napster. Napster was one of the first P2P networks in widespread use. It was mostly used for music sharing. In Napster only the actual file-transfer was initiated between two peers, the search was handled by a central server (see Figure 2.1(a) on page 12).

A second, modern P2P protocol widely used today and employing a central server is BitTorrent [8]. BitTorrent is a protocol especially suited to efficiently distribute large files or collections of files. Nodes that want to download content via BitTorrent connect to the responsible tracker. The tracker knows the IP-addresses of all other nodes that are currently downloading or seeding the content and acts as a rendevouz point [25, pp. 2].

[1] This spike of popularity is often called a “flash crowd” or the “slashdot effect” [7, p. 1]. For a number of other reasons why usage spikes can occur see [7]

P2P systems with a central server have got several advantages over the traditional client-server system. Bandwidth is usually no longer of concern, the problem of a single-point-of-failure still exists. The systems don't work without their respective central servers. But why do these systems use central servers for certain operations and don't handle everything via Peer-to-Peer traffic?

There are speed and complexity advantages for certain operations if you do not use a fully distributed protocol. In Napster the central server was used for searching – when a search query arrived, the server could tell straightaway whether the file searched for is present in the network and where exactly it can be found. This is a big advantage that cannot be realized in this way without a central entity.

### 2.1.2. Pure Peer-to-Peer Networks

Pure P2P networks without a central server surfaced a short time after Napster [12, p. 42] appeared. Examples for protocols these P2P networks use are the Gnutella 0.4 [18] or the Freenet protocol (see section 4.3.1 on page 29). The typical hierarchy of such networks is shown in Figure 2.1(b) on page 12.

Because these systems work without any central servers there are certain complications in comparison to server-driven P2P networks. To join the network a new node has to know another node that in turn already is part of the network. But the number and addresses of the systems that are part of the network are in constant flux. Typically, this problem is solved by releasing lists with many nodes that were in the network at a specific moment of time. A new node then works down the list until it succeeds in connecting to one of the systems.

Searching for data without a central server also poses a problem. Because there is no central entity knowing which data is stored on which node, searching in unstructured networks is typically realized with a flooding mechanism. The search query is simply forwarded to all or some neighbours which in turn forward the query until the specified piece of data is found or the query packet's time-to-live expires.

This is not a reliable way to search for information. Same search queries may (and often will) give different results if they are run multiple times because they took a different way through the network each time.

### 2.1.3. Hybrid Peer-to-Peer Networks

To compensate for the mentioned problems of pure P2P networks, hybrid P2P networks were the next evolutionary step. Examples for networks using hybrid P2P protocols include Gnutella 0.6 [30], eDonkey2000 [14], FastTrack (most prominently used by Kazaa [28]) or Skype. These network protocols try to combine the advantages of pure P2P systems with those using a central server.

In hybrid P2P networks certain well-connected servers get a special status (the special nodes are called supernodes or superpeers or similar). These supernodes are well interconnected to each other and are responsible for a certain number of other "normal" nodes.

Supernodes handle certain tasks that are more suited to be handled centrally. For example searches can be handled more efficiently, when a supernode is present: each node sends its list of available files to its supernode. When a node searches for a file it sends the search query to its supernode. The supernode forwards the query to the other supernodes and the query can now be answered without consulting any "normal" node. This is a much more efficient way than in pure P2P networks – but also opens up certain vulnerabilities not present before. One can e.g. try to block the access to the supernodes (which are relatively limited in number) and therefore diminish the speed or usability of the P2P network [see 39].

A typical hierarchical P2P structure is shown in Figure 2.1(c) on the next page.

### 2.1.4. Distributed Hash Tables

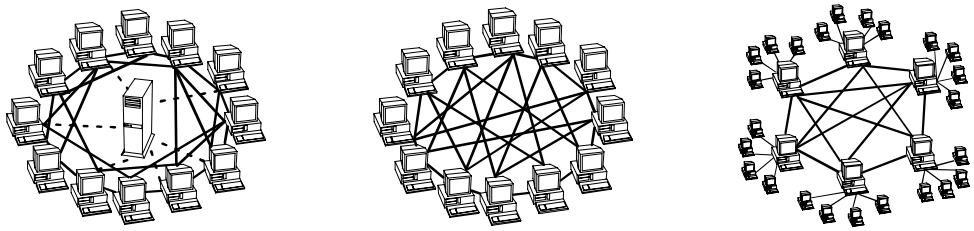
Now that we have a knowledge of the different types of unstructured P2P networks we take a look at structured P2P networks. Structured P2P networks take a different approach to the routing problems. They use Distributed Indexing, most often in the form of Distributed Hash Tables (DHTs) [62, p. 84] to avoid the lookup problems described earlier.

The basic idea behind a Distributed Hash Table is, that a unique identifier is assigned to every piece of data. Often this identifier is derived by applying a collision-resistant hash function like SHA-1 [11] to the data in question [62, p. 85]. Each node of the network node also gets an unique identifier in the same address space and is responsible for a certain interval of addresses near its node address. The exact interval a node is responsible for varies: a few P2P systems assign all identifiers that are larger than the node ID but smaller than the ID of the next node to the node in question. We assume that each node is responsible for all identifiers that are numerically closest to its ID. The node-identifiers are usually computed by hashing certain pieces of information that are specific to the node. E.g. Chord [56] computes the node identifier by hashing the node's IP address [56, p. 3].

Figure 2.2 on page 15 shows a network with 8 nodes and an address space of 10 bits (1024 addresses) ordered in as a ring. We assume that the node with the nearest identifier is responsible for a certain address. In our example if we want to look up a piece of data with the identifier 100 the node with the number 147 would be responsible. We would have to ask this specific node for the piece of data.

That means if we are node number 952 and we have got a piece of data with an associated identifier of 100 which we want to publish in this P2P network, we first have to send it to the node 147. Everyone else then can request the information from these nodes.

In most cases this approach is unpractical - transferring data to another node takes time and we don't even know if any node will ever request the data. Owing to this fact, a single



(a) Centralized (b) Pure (c) Hybrid

Figure 2.1: Different types of P2P Networks [from 12, p. 36]

layer of indirection is used in many cases. In our example, we once again want to store a piece of data with the identifier 100. But this time we take a different approach: we only store a pointer at node 147 which contains the information that the actual data record is stored at node 952 (because that is our node ID).

This approach has certain further advantages: let's assume that node 421 wants to download the piece of data with the identifier 100. It requests the data from node 147 and receives the reply that the actual data is stored at node 952. Now it can get the data from node 952 and then tell node 147 that the data is now present two times in the P2P network - one time at node 952 and one time at node 421.

When a new node joins the network it assumes responsibility for the nearest identifiers. The nodes that previously were responsible for the identifier ranges affected send that data to him. When a node leaves the network, it sends the affected data to the nodes that will be responsible for it after it left. With this technique only very few nodes are affected when a new system joins or leaves the network. This approach is called "consistent hashing" and was presented in [27, 36].

Now we have examined how we store and address data in a DHT. What we haven't talked about untill now is how the routing algorithms work. The different DHT systems use different routing strategies. We will show the routing algorithm that was chosen for IGOR in section 2.2.2 on page 14. In principle the algorithms are all a little bit similar. In every algorithm all nodes make use of some type of routing table that is utilized to get each message to its destination address as quickly as possible.

But Distributed Hash Tables also have got certain disatvantages. They provide an efficient means of routing data accross the network and lookups are guaranteed to yield an exact result. But all this has got a price: the most notable shortcoming of structured Peer-to-Peer networks is the absence of fuzzy searches. These are very easy to implement in unstructured P2P networks, but a big challenge in structured ones which usually only allow the lookup of exactly identified pieces of information.

## 2.2. The IGOR overlay network

The IGOR File System which was modified in this paper uses the IGOR overlay network for communication; because of that we will give a brief overview about it. IGOR is a key based routing system and in many ways similar to e.g. Chord [56] or Kademlia [40]. As already mentioned in section 2.1.4 on page 11 IGOR is a structured overlay network. It acts very similar to other structured Peer-to-Peer networks and we will show some of the distinctive features of the IGOR overlay network in this section.

### 2.2.1. Features

In IGOR the network has got an 160-bit address space, hence $2^{160}$ addresses exist. Every node of the network acts as a server and as a client at the same time. Each node is identified by one address from the address space. Each node is furthermore responsible for the address space that is near its own address. If you have e.g. two nodes; one node $A$ with address 0 and one node $B$ with address 10 and a message with the destination address 7, it will be routed to $B$; a message with the address 4 would be routed to $A$.

One can imagine the whole address space as a big circle where all messages with a certain destination always arrive at the node with the nearest number. This is illustrated in Figure 2.2 on page 15, but with a 10-bit address space instead of an 160-bit address space. IGOR creates a ring topology, that means that the node with the highest identifier is a neighbour of the node wit the lowest identifier, in the picture this would be the nodes 14 and 952. It was already mentioned, that in IGOR, nodes are responsible for all addresses that are in closest proximity to them; e.g. node number 14 would be responsible for the addresses 995 to 27. If a number is on the exact middle between two nodes, the node with the lower node identifier is responsible for it.

The IGOR network can deliver messages in $O(\log N)$ hops while maintaining connections to $O(\log N)$ other nodes ($N$ being the number of nodes in the network) [33]. This makes the IGOR network very scalable. In the IGOR network every message carries an application or service identifier besides the destination address. IGOR delivers messages only to nodes where an application with the given service identifier is registered. This allows multiple applications to run on one Peer-to-Peer network without interference.

The applications that use IGOR connect to IGOR via TCP/IP [51]. This makes it possible, that IGOR itself runs on a host that is placed in a demilitarised zone (DMZ), where it can easily connect to other IGOR machines outside of the local network. The application host itself can be within the firewalled internal network; it only has to be able to establish outgoing TCP connections to the IGOR host.

Igor features proximity neighbour selection PNS [33, 19, 29], that means it adapts its topology to the characteristics of the underlying physical network. It also uses proximity route selection [33, 19, 29] to use the overlay network in an efficient manner.

At the moment a distributed video recorder [32] and a SIP-proxy [15] have been imple-

mented besides IGOR-FS [33] itself that we have extended in this paper.

### 2.2.2. Routing

We will now give a short overview over the routing algorithm used in IGOR. We already established, that each IGOR node knows its direct neighbours. But it is quite obvious, that this is not fast enough, with only this information, we would need $O(N)$ hops.

Because of that, every node has got a routing table called the "finger" table. This table simply contains node-ids and the IP-addresses of these nodes. The size of the finger table is dependent of the number of participants $N$. In IGOR it has got at most 160 and usually $log_2 N$ entries [29, sec. 2.4.2].

The finger table must be filled cleverly to reach the logarithmic hop count. To accomplish this goal, the table is filled in a way, that gets exponentially more fuzzy. A node knows many nodes in its neighbourhood, as you move farther away the information gets more imprecise. This is actually quite logical, because now a message can be routed quickly to a node, which has got an ID that is relatively near to the ID of the actual target node. And this node knows in more detail where the target node can be found, until the right node is finally reached.

In detail the ID-distance between two nodes $n$ and $i$ in an m-bit address space is defined by the formula [see 29]:

$$d(n, i) = |n - i| \bmod 2^m$$

A node entry belongs into the $i$-th finger of the finger table if the ID-distance is in a certain range:

$$d\left(n, \text{finger}(i)\right) \in \left[2^i, 2^{i+1}\right] \text{ with } m < i \leq 0$$

As one can see, the ID-distance between the node $n$ and the entries in the finger table grows exponentially for entries with a higher number.

## 2.3. IgorFs

The IGOR File System is one application that uses IGOR as its underlying network. To the user IgorFs simply appears as a file system that can be mounted in Linux. IgorFs organises the data that is stored in it with a Distributed Hash Table. That means, that every data-block that is stored in IgorFs is accessible via its identifying hash key.

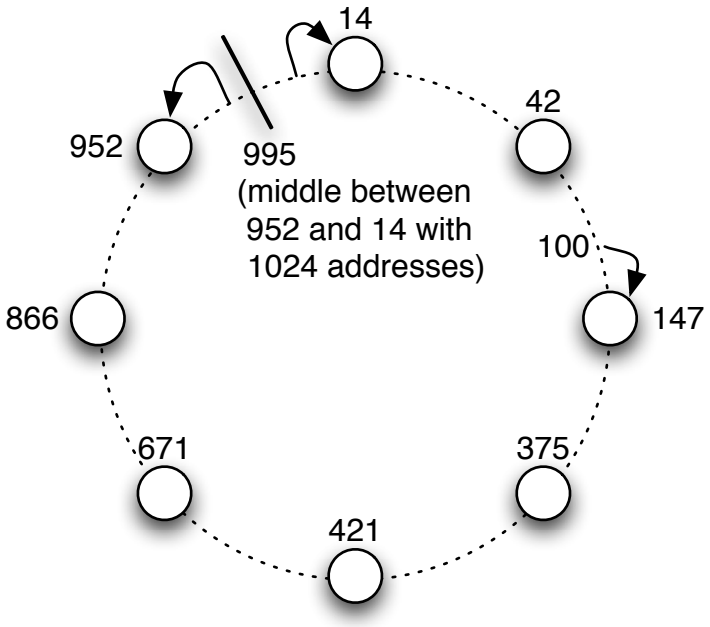



Figure 2.2: IGOR address space

### 2.3.1. FUSE

Traditionally, operating systems distinguish between the "kernel" of an operating system which is responsible for things like scheduling, interfacing with the hardware, memory management, etc. and between the user programs [see 59, p. 671]. Usually the kernel is also responsible for handling file systems and only exposes a fixed set of operations to the applications like opening or closing a file, reading data from or writing data to an it...

FUSE (File System in User Space) is a project making it possible to write file systems that run entirely the user space.

Usually when a user program tries to access a file, it uses a system call to accomplish this. This system call is forwarded to the VFS (Virtual File System) layer of the kernel, which in turn converts the system call to the right format and forwards it to the correct file system driver for the affected file system (like e.g. the ext3 or ufs driver). In principle FUSE just acts like another file system driver to the VFS layer of the kernel.

When a request arrives at the kernel part of FUSE, it simply is forwarded to the fitting user-space process (in our case the IgorFs process). This process in turn has to send an answer back to FUSE. A sample request is shown in greater detail in Figure 2.3 on the following page.

In this figure we assume, that a user is issuing a list command in his shell and wants to

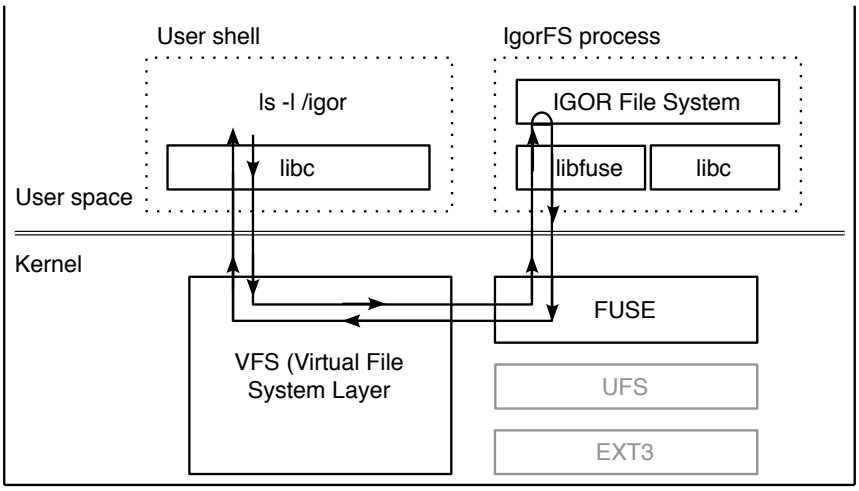



Figure 2.3: FUSE sample request [see 59]

see the contents of a directory that is part of IgorFs. The shell executes the fitting function of the standard C-library "libc". The C-library creates a system call, which is sent to the VFS layer of the kernel. The kernel VFS layer forwards the request to FUSE, from where it is sent to the IgorFs application (via the linked C- and Fuse libraries). The application sends the request back the same way it arrived and finally the kernel sends the answer to the user program.

### 2.3.2. Block Handling and Encryption

All files that are stored in IgorFs are first split into blocks. Unlike regular file systems that use physical media, IgorFs does not use fixed block sizes, the block size is calculated using a rolling checksum [see 33, sec. 33]. Because of this even if a change occurs in the middle of a file, most of the blocks do not change. In IgorFs it is e.g. a trivial operation to remove the first few bytes of a file; for most other file systems that means that you have to re-write every block of the file system, because the block size is constant and you cannot simply remove a few bytes from a block.

After a file has been cut into blocks, the individual blocks are encrypted with their own hash key. That means, that first the block $B$ is hashed with the hash function $H$. Therefore the encryption key $k$ is $k = H(B)$. The encrypted block is created by applying the cryptographic function $E$ with the key $k$ to the Block $B$. The encrypted block is therefore $E_k(B) = E_{H(B)}(B)$.

This block is then inserted into the DHT and is identified by $\mathrm{ID}(B) = H(E_k(B)) = H(E_{H(B)}(B))$. This way of storing the blocks enables IgorFs to check the integrity of a

block that was retrieved; it simply can check if the hashed data is equal to the requested id. The decrypted data can be checked in a similar manner.

To successfully retrieve a block from IgorFs, one needs two pieces of Data: the identifying hash ID($B$) and the cryptographic key $E_k(B)$. Together these two values form the tuple (ID, $k$).

### 2.3.3. Directory Handling

As already explained in the last section, files are made up of one or (mostly) several blocks. Directories are made up of files, and are also stored as blocks in the DHT. The directory blocks contain the (ID, $k$)-tuples for all the blocks of every file in the directory.

Anyone who knows the (ID, $k$)-tuple for a directory can read that directory. If one knows the (ID, $k$)-tuple of the root directory of a file system one can read the entire file system.

When the files in the file system change, their (ID,$k$)-tuples also change. The new directory entries are periodically written to the DHT, again with a new (ID,$k$)-tuple. That means, that the tuples change for every directory that was changed, hence they are propagated downwards the directory tree until they reach the root-directory.

In this paper we want to find a way, to distribute the new (ID,$k$)-tuples for the root-directory in a secure manner to all authorised nodes.

### 2.3.4. Block Transfer and Caching

In IgorFs blocks are not really stored directly in the DHT. Instead we use the indirection approach mentioned in section 2.1.4 on page 12. When a new block $B$ is inserted into the DHT, it is stored on the local node only and a pointer is inserted into the DHT at the address ID($B$) (this is called the “Pointer Store”).

When a node requests a data block, it first looks up the real location of the block in the Pointer Store. After that, the node requests the block from the real location and delivers it to the operating systems VFS layer. But the local IgorFs also caches the block and tells the Pointer Store that the block now also can be retrieved by querying us.

Consequently blocks that are popular and requested by many nodes are also broadly available.

# 3. Publish/Subscribe Systems

Now that we have covered the basic principles of Peer-to-Peer networking and IgorFs, we will take a more thorough look at our problem and evaluate the approaches that were taken to solve similar problems.

Our problem (in a very short form) is, pushing updates to authorised clients. In general systems with a publisher and (many) clients are called "publish/subscribe systems". There already are a number of different system available, many of them are very widely in use today.

One additional requirement for our solution is, that it has to be secure. The data that is exchanged will be propagated via the Peer-to-Peer network and thus has to be encrypted in some way. Otherwise every node that forwards the data could (if it intercepts the correct piece of data) access our file system.

## 3.1. Centralised Publish/Subscribe Systems

Traditional publish/subscribe systems are implemented in a centralised manner, that means that there is one central server and all clients poll this server regularly for new information. A popular and widespread example for this are RSS [34] and Atom [50] feeds.

RSS and Atom feeds both work in the same way. An XML [68] file containing information about e.g. news articles is published on an HTTP server. All clients poll the file in regular intervals and look if it has changed.

Centralised systems are easy to implement and to understand, but they have got several shortcomings which cannot be remedied easily. The most important shortcomings are the lack of scalability and fault-tolerance. In principle the same problems that were already discussed in section 2.1 on page 9 also apply here.

## 3.2. Multicast

With the conception of Distributed Hash Tables (DHTs) there have been several new proposals on how to implement distributed publish/subscribe systems. Most use some kind of multicast within the overlay network to distribute the new information to the subscribed participants.

Such a system is proposed in [57]. In this paper the authors propose using Scribe, an application-level multicast system [4] to transmit their notifications. Scribe is a distributed multicast system, which is built on top of Pastry [52], a DHT System developed at Rice University. In Scribe to join a multicast group, a subscriber needs only two pieces of information: the name of the multicast group they wish to join and the name of the creator of this group. No other information is needed. Scribe supports large numbers of groups, with a potentially large number of members per group [see 4, p. 1489]. It also balances the load on the individual nodes.

In Pastry, each node has got a unique, 128 bit node ID. The node IDs are uniformly distributed. Each pastry node is responsible for a fraction of the 128 bit key space: it manages all keys that are numerically closest to its own node ID.

In Scribe each group has a unique group ID. The group ID is the hash of the group's textual name concatenated with its creator's name. The hash is computed using a collision resistant function like e.g. SHA-1 [11] which at the same time ensures a uniform distribution of the group IDs. The pastry node with the closest node ID to the group ID becomes the rendevouz point for the group. The multicast tree is created with a method that is very similar to reverse path forwarding [9].

While all this sounds very good there are several problems with this approach. In our case the information that is transmitted needs to be encrypted. Otherwise every node that forwards the update notifications would be able to read the entire file system. The traditional solution to this problems is an encrypted multicast. Solutions to this problem have been shown in [61] and [66] almost simultaneously. In these papers a hierarchical approach to key management in multicast systems is shown. Using this approach the key management overhead is relatively small while full forward and background confidentiality can be guaranteed. The biggest problem in an encrypted multicast system is forward confidentiality. After a participant has left the multicast tree it should no longer be able to decrypt any messages that are distributed within the tree. This means that the multicast tree key has to be changed in a way that the participant who has left the tree cannot deduce the new key from intercepted data.

The solution to this problem is quite complex. The authors of the two papers build a hierarchy of keys hat allows a secure and relatively fast re-keying of the multicast tree. Every node gets several keys and some of these keys are distributed to sets of nodes. If a node leaves the tree we can send messages that are encrypted with the set-wide keys to the multicast tree and re-key whole sets of users at the same time with them. If we choose the correct keys, this re-keying operation is relatively efficient and the node that left the tree cannot decode any new message.

In contrast to typical encrypted multicasts, we would not need full forward confidentiality in our case. A participant that leaves the multicast tree may usually still rejoin the tree at a later time (if it wishes to do so). The only case when it may not rejoin the tree is when his access privileges to the file system are permanently revoked. That means that in our case we would only need to re-key the multicast tree if access privileges for a node are

revoked.

But this approach has once again an inherent problem: the need for a central authority that controls and distributes the (hierarchical) encryption keys of the multicast tree. This central authority would need to always be present, otherwise a key hierarchy could not be established (the hierarchy is needed in case of the revocation of the access rights of an individual). And without a key hierarchy the complexity of a re-key of the tree would be $O(N)$ (with $N$ being number of participants). This is an unacceptable restriction of this approach.

## 3.3. Multicast without Global Key

A second approach to this problem would be an encrypted application-level multicast tree without a central key. Instead every link between two nodes of the tree has got its own key on which the nodes agree autonomously. This approach has got several key advantages to an tree with only a single key: after a node leaves the multicast tree it can under no circumstances decrypt any data sent within the tree, it also can never decrypt any data that was intercepted before it joins the three. This means that we do not have to worry about forward and backward confidentiality. A disadvantage of this approach is, that the messages that are distributed via the tree have to be decrypted and re-encrypted for further forwarding at each node of the tree.

However this should not post a problem in our usage scenario. We only want to distribute the new IgorFs (id, key)-tuple. This means that we only have got a very small amount of data that is transferred via the multicast tree. Re-encrypting should not pose a significant strain on the system resources of the tree members.

To identify the clients we use X.509 certificates [23]. This allows us to build the multicast tree without the presence of a central server.

### 3.3.1. X.509 Public Key Infrastructure

X.509 certificates are commonly used in the Internet today. Their most widespread use is to securely identify web sites.

A X.509 certificate is used to securely identify a individual person or a service in the Internet. A certificate contains (among other things) the public key of the certified person, the name of the person and the expiration date [13, p. 390]. It is signed by a Certificate Authority (CA).

To verify the identity of a person or service one just has to verify if the certificate is signed by a trusted CA. This is mostly transparent to users. Modern operating systems usually have a list of trusted CA's which is installed automatically. Many big enterprises or universities also have got their own CA. An example would be the DFN-PKI PCA

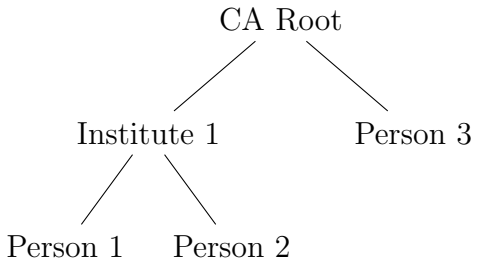



Figure 3.1: Certificate hierarchy

(Deutsches Forschungsnetz-Public Key Infrastructure Policy Certification Authority [10]) of Germany's national research and education network.

Certificates do not have to be signed directly by the Certificate Authority. Instead they can be signed by an Intermediate Authority which in turn is signed by the CA. This allows a delegation of the certification process.

A practical example for this would be the following situation: We are publishing our own file system, have created our own CA and already have issued a few certificates to people who may access our file system. Now a whole faculty of a university shall be allowed to access our file system. In this case we do not create a certificate for each member of the faculty. Instead we give a signed certificate to the faculty which they may in turn use themselves to create certificates for their user. When the faculty should no longer be allowed to access the file system we only have to revoke the one certificate we gave to the faculty.

### 3.3.2. Key Exchange Protocol

What we need now is a way to exchange the link keys when someone wants to join the multicast tree and a way to detect if someone is allowed to join the multicast tree without the need of a present central authority. Our solution is based on the forementioned X.509 certificates. Everyone who wants to operate a file system first has to create a certificate authority (CA). We call a file system operator a publisher. Everyone who may access the file system gets a X.509 certificate that is signed by the CA of the publisher and the extended key usage set to "clientAuth" [see 23, sec. 4.2.1.13]. Everyone who may distribute a new file system hash and key gets a X.509 certificate that is signed by the CA and has the SSL extended key usage set to "serverAuth".

To join the tree we first have to find a node that already is part of the tree. To accomplish this, we could use a method similar to that of Scribe. We use the multicast group name to generate an Igor identifier and route our messages to the rendevouz node. A node that wants to join the multicast tree needs a valid client certificate and the certificate of the CA. The certificate of the CA is needed to allow the client to check if the node it is connecting to

has got a valid certificate. The address of the rendevouz node is generated by hashing the CA certificate. The node that wants to join the tree sends a join-message to the rendevouz node. The first node that already is part of the tree and gets our message and replies to it in the way described below.

A problem can arise, if our multicast tree is very sparse. A join-message could arrive at the rendevouz-node itself without first arriving at a node that already is part of the tree. But the rendevouz-node itself does not have to be part of the multicast tree and we therefore cannot join the multicast tree directly at this node.

The solution to this problem requires cooperation of the rendevouz node. A malicious rendevouz node could render this whole approach useless.

One possible solution would be, that the publisher signs his own address and sends it to the rendevouz node. The rendevouz node can verify the identity of the publisher by verifying that the hash of the CA points to him and that the message is signed by a server certificate. When a join-message arrives at the rendevouz node, the node simply changes the destination address to the address of the publisher. In the worst-case the packet will arrive at the publisher itself, who can answer it.

When we have found a node that is part of the multicast tree, we have to join the tree in a secure way. The key exchange protocol I propose is similar to the one first proposed in [43] with the extension of nonces ([44], [13, pp. 416]). Unlike in [44], we use the nonces not only to prevent replay attacks of the key exchange. We also use them as the initialisation number for the sequence counters we use for message exchange.

Let $A$ be the participant who wishes to join the tree and $T$ the nearest node that already is part of the tree.

The first message is sent from $A$ to $T$ and contains our nonce $I$ and the certificate of $A$ $\mathrm{cert}_A$.

$$A \longrightarrow T \qquad \{\mathrm{sign}_A(I), \mathrm{cert}_A\} \tag{3.1}$$

$T$ now validates $A$'s certificate and the signature of the sent data packet. If the signature and the certificate are valid T replies with a signed and encrypted message that contains $A$'s nonce $I$, his own nonce $J$, his own certificate and the symmetric encryption key $K$ that is used for the remainder of the communication between the two nodes.

$$T \longrightarrow A \qquad \{\mathrm{enc}_A(\mathrm{sign}_T(I, J, K), \mathrm{cert}_T)\} \tag{3.2}$$

$A$ decrypts the message with his secret key, gets the certificate, validates the certificate, checks the signature and the nonce $I$. If everything is OK, $A$ can be sure that node $T$ is who it claims to be. $A$ now replies with a packet that contains $J$ to verify that it has got the packet. This packet can already be encrypted with the symmetric cipher.

$$A \longrightarrow T \qquad \{\mathrm{enc}_K(J)\} \tag{3.3}$$

This key exchange protocol has got some similarities to the three-way-handshake mechanism that is used to establish TCP [51] connections.

### 3.3.3. Message Exchange

The protocol we use to exchange messages between our nodes also is similar to TCP.

Our goal is to distribute the new (hash, key) tuple to the entire multicast tree. When a new (hash, key) tuple needs to be distributed, the publisher sends a message to the multicast tree that contains the new values and his certificate. This message is signed with his secret key. Consider the situation where the publisher $P$ is connected to the multicast tree via a node $A$. The connection between $A$ and $P$ is encrypted with the key $K$. This is the first message exchanged between $A$ and $P$ after the initial key exchange and $A$ used the nonce $J$. $T$ is a file system revision number. It is important that the revision number is increased each time. A simple method to do this would be to use an Unix timestamp.

$$P \longrightarrow A \qquad \{\mathrm{enc}_K(J+1, \mathrm{sign}_P(\mathrm{hash}, \mathrm{key}, T), \mathrm{cert}_P\} \tag{3.4}$$

Node $A$ replies with the sequence number to verify that it has got the packet. This is somewhat similar to a TCP ack.

$$A \longrightarrow P \qquad \{\mathrm{enc}_K(J+1)\} \tag{3.5}$$

Now node $A$ checks the validity of the attached certificate. $A$ also checks that the certificate is an SSL Server certificate, and that it has not already updated the file system revision with a higher revision number (to prevent replay attacks). After that $A$ checks the signature of the IgorFs hash and key. If the signature is valid $A$ sets the file system root to the new values. Now $A$ forwards the message to all multicast nodes that are connected to it (with the exception of the node that it received the message from). The messages sent to the nodes have the same content as the message $A$ received from $P$. Only the encryption key $K$ and the sequence numbers have to be recomputed. The signed part of the message and the attached certificate remain untouched.

Now every node that got a message from $A$ can verify the validity of this message using exactly the same steps $A$ used.

### 3.3.4. Certificate revocation

In section 3.3.1 we mentioned, that certificates of people or institutions who no longer may access the file system have to be revoked. But how can we check, if a user who tries to join the multicast tree is revoked, without having to consult a central entity?

The solution to this problem is stunningly simple: we are in a network that implements a DHT. When a user $u$ is revoked by the publisher $p$, we create a note that his certificate is revoked. We sign this note with the publishers certificate and store the resulting message at the address we get by hashing the fingerprint $F(u)$ of the users certificate.

$$H(F(u)) = \text{sign}_P(\text{rev}(u))$$

Now when $u$ tries to join the multicast tree at node $A$, $A$ looks if a note exists at address $HF(u)$). If it exists, the validity is checked and $u$ is not admitted to join the tree.

Naturally when a user is revoked the multicast tree also has to be notified of this and if the user is at the moment active in the multicast tree he has to be disconnected.

# 4. Subset Difference Revocation Scheme

Until now, this paper described the mostly traditional solutions to our problem. The techniques used there probably will sound familiar to people who have an interest in Internet security. In this chapter we will take a look at the approach that was chosen by us in this paper, which takes another way to solve the problem.

In 2001 D. Naor et al. proposed a new revocation scheme called the "Subset-Difference" (SD) revocation scheme [42]. This scheme allows a publisher to send an encrypted message that every authorised receiver but no revoked receivers can decrypt. To be more exact, if $\mathcal{N}$ with $|\mathcal{N}| = N$ is the set of all users and $\mathcal{R} \subset \mathcal{N}$ with $|\mathcal{R}| = r$ is the set of revoked users, the publisher is able to transmit a message $M$ to all users, so that any user $u \in \mathcal{N} \backslash \mathcal{R}$ can decrypt the message, but no user of $\mathcal{R}$ or even a coalition of all users in $\mathcal{R}$ can decrypt it. [42, p. 6]

The receivers are stateless, they do not have to save any information. The scheme requires the receivers to store $\frac{1}{2}\log^2 N$ keys, the message length is at most $2r$ keys and should be much shorter in our case. The algorithm guarantees complete security even if all revoked users collude their keys [38, p. 1]. The computing requirements are $O(1)$ for the receivers and $O(r)$ for the sender.

This algorithm is used in the new HD and BluRay DVDs for example. The encryption system used there is called Advanced Access Content System (AACS) [1] and has become rather infamous recently after it was reported broken [58]. But these reports have been misleading: there is no known weakness in the algorithm itself. Some people managed to retrieve the encryption keys from the memory of their DVD players and used them to decrypt the content themselves. This is no weakness of the revocation scheme but only a weak point of this specific usage of the system.

It was because of these media reports, that we discovered that the encryption scheme could be a viable alternative to the classical multicast encryption. After reading about the problem the author of this thesis became interested in AACS and after reading the specification and several papers he realized, that it could be used to generate a good solution to the problem of this paper.

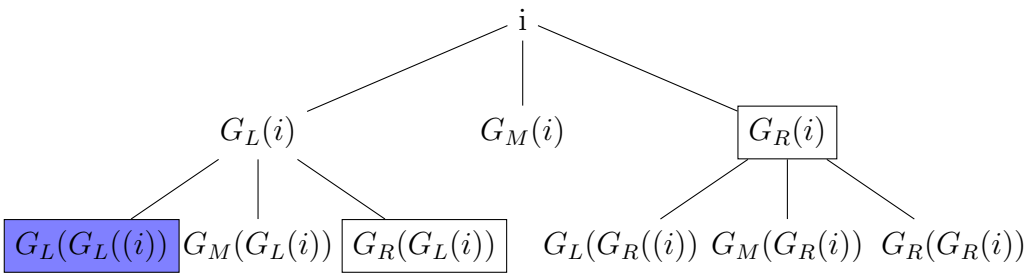



Figure 4.1: Example tree built from $i$

## 4.1. How does it work

The Subset Difference revocation scheme heavily relies on a pseudorandom number generator $G$ that takes an arbitrary number as input and calculates a number that is exactly three times its length. If we take the number $i$ as input for the generator $G$ we name the left third of the output of the generator $G_L(i)$, the middle part $G_M(i)$ and the right part $G_R(i)$. $G$ has to be a one-way-function, it must not be possible to calculate $i$ from $G_L(i)$, $G_M(i)$ and $G_R(i)$ even if you have all three of them.

The second thing you need is a symmetric encryption function $F_K(T)$ that takes a key $K$ and encrypts a string $T$ with it. $F_k$ should be fast and and should not expand the plaintext [38, sec. 2.2].

With the help of our generator $G$ starting with a initialisation value $i$ we can build a tree with an arbitrary height in which every node is identified by a (unique) number. An example for a tree with height 3 is shown in Figure 4.1.

If you know the initialisation number $i$ (and the algorithm $G$) you can calculate the number of each node in the entire tree.

To use this tree for the revocation scheme we have to distribute certain numbers to our participants. Every leaf of the tree represents a participant. Now let's assume for a moment that we are the participant at the lower left of the tree in Figure 4.1 (identified by the "blue" node $G_L(G_L(i))$). Our objective is to find a method that allows us to encrypt messages in a way that doesn't require the participants to save a state and cannot be decrypted by revoked users.

We are the blue user. We only have knowledge of the keys $G_R(i)$ and $G_L(G_R(i))$. With these two keys we can calculate the identification number of each node in the tree except our own one and those of our predecessors.

We distribute the keys to the other participants in exactly the same way. Now everyone can calculate every key in the tree except the keys of himself and his predecessors (which in this case only is $G_L(i)$).

If someone decides to encrypt a message that everyone but us shall be able to decrypt

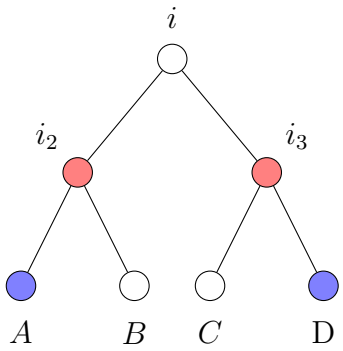



Figure 4.2: Tree with two revoked users

he can simply use $G_L(G_L(G_M(i)))$ as the message key $k$ and send the information that the message is encrypted with the path $L, L$ as well as the encrypted message $F_k$(Message). Everyone in the tree but us can calculate the key. If someone wants to send a message that only members of the right subtree of $i$ can decrypt he simply encrypts it with the key $G_L(G_M(i))$.

But what do we do, if we want to revoke two participants at the same time? (e.g. $A$ and $D$ in Figure 4.2)? We cannot do that at the moment. If we encrypt the data with the key of $A$, $D$ still can decrypt the data and vice versa. The only solution would be to encrypt the data with $i$. Neither node knows this value. Unfortunately this would mean that $B$ and $C$ also cannot decrypt the data.

The solution to this problem is quite intuitive: we expand the encryption scheme and use a different initialisation number for every subtree of our tree. That means that we now have our central "master tree" in which every node has a unique key associated with itself. But at the same time every node of this central master tree is the root node of a new tree, that uses our generator $G$ to create the keys for all its nodes. If you want to imagine this visually, think of a tree that is laying on a plane. At every node a new tree is rooted that grows downwards from this plane.

In our example we have the main tree with the initialisation number $i$, the left subtree with $i_2$ and the right subtree with $i_3$. The key distribution in the subtrees works in exactly the same way than in the original tree. That means that now our participant $A$ gets the following numbers: $G_R(i)$, $G_L(G_R(i))$, $G_R(i_2)$.

In our example we now take the left subtree with the key $i_2$ and encrypt our data with $G_L(i_2)$. Furthermore we take the right subtree with the key $i_3$ and encrypt our data with $G_R(i_3)$. Now $B$ can decrypt the message that is encrypted with $G_L(i_2)$ and $C$ can decrypt the message that is encrypted with $G_R(i_3)$. Neither A nor D can decrypt the message.

What we have now are two subsets differences. The subsets are the nodes of $i_2$ and $i_3$ respectively, the differences are the nodes of $A$ and $D$.

Because we do not want to encrypt the entire message twice we encrypt the message with a master key and only encrypt this master key twice. Each authorised client can decrypt the master key and use it to decrypt the message.

The number of keys every participant has to store is $\frac{h(h+1)}{2} + 1 = \frac{1}{2}\log^2 N + \frac{1}{2}\log N + 1$ ($h$ being the tree height and $N$ being the number of nodes in the tree). We chose a tree height of $h = 32$. With this tree we can support up to $2^{32} - 1 = 4294967295$ users (at least one user always has to be revoked for the scheme to work). Each user in this tree has to save 529 keys. With a key length of 128 bits, that amounts to 8464 bytes of data.

## 4.2. Encrypted Multicast

After presenting the Subset Difference method that has some very interesting properties we now show a way to use it that solves our problems.

We want to distribute short messages to a (potentially) big number of receivers. We have found a way to encrypt our messages such that only legitimate receivers can decrypt it. Hence we can simply publish our message via a multicast channel to which everyone may subscribe with a system like e.g. Scribe [4]. Everyone who is authorised can decrypt our message. Everyone else cannot decrypt it and there is no harm done if they get the encrypted message.

This distribution scheme would be easy to implement and should be fast.

One problem with this approach is backwards-confidentiality. With the naive implementation of our revocation scheme, there is no backwards-confidentiality, everyone who has a valid set of keys can decrypt every message that was sent, even if he was not part of the tree at the time, the message was sent.

This should not pose a problem for our specific use of the encryption scheme. Usually everyone who has access to the file system may access every revision of the file system, even older ones – and there should be no harm done by this. But there may be cases, where it is not desirable, that someone who just got his set of keys can access every old revision of the file system (if he captured the old encrypted packets, that were distributed via the multicast tree). Perhaps there was some confidential data present in the old file system revision, which we don't want the new node to get knowledge of.

There is no trivial solution to this problem. One possible solution would be to start with a subset-difference-tree, where every node is revoked and only allow clients to decrypt the data, when their keys are sent to them. But this is only feasible if the paths of the nodes are issued in a consecutive manner. Else the size of the encrypted message will increase considerably after a very short time.

Forward confidentiality should be no problem with our approach - a user who no longer may access the file system is revoked and can no longer decrypt messages, that were sent via the tree.

## 4.3. Pull Mechanism

A second solution we propose, would use a pull mechanism rather than a the multicast push mechanism.

With IgorFs we already have a method for efficient data storage and replication. With the Subset Difference revocation scheme we have an encryption method for stateless receivers. We could combine these two things and simply store the encrypted (hash, key)-tuple in IgorFs.

Storing our data in IgorFs offers several key advantages: by storing the data in IgorFs we can minimise the complexity of our solution. Furthermore IgorFs implements a block caching algorithm. Our encrypted (hash, key)-tuple will automatically spread itself across many nodes if it is frequently requested.

But simply storing the encrypted (hash, key)-tuple in IgorFs will not solve our problems. The block in which we store the encrypted data will itself be identified by a (hash, key)-tuple and we would need to distribute this data to everyone who wants to access our file system. Naturally we could (once again) distribute this via an (unencrypted) multicast.

What we would need is a way that enables all subscribers to predict the hash and key that is used to store our encrypted data. That way everyone could get the data without a multicast tree or any other prior communication.

### 4.3.1. Freenet

We will take a short look at another system that supports a method to access data that is very similar to what we need:

When you look at Freenet up to version 0.6[1] [6, 5] there are several different types of keys to access the stored data. The two most important and most interesting ones for us are CHKs (Content Hash Keys) and SSKs (Signed Subspace Keys).

Note that Freenet is not using a DHT to store its data. The data in Freenet is addressed by hashes, that look similar to DHT hashes and also have a similar function. But the communication is totally different. Data is searched with a flooding-algorithm.

A CHK is a key that identifies static content in Freenet. A typical CHK key looks like this [16]:

```
CHK@file hash, decryption key, crypto settings
```

This is very similar to the (ID, key)-tuple IgorFs uses to access data stored in the DHT.

Signed Subspace Keys are used for data going to change over time. SSKs use public-key cryptography. Only someone who knows the private key may add a new version of the file.

A typical SSK key looks like this [16]:

```
SSK@public key hash, decryption key, crypto settings/name-version
```

[1] Freenet 0.7 is a complete rewrite and works quite different. A darknet[53] approach was taken there.

The file hash key is computed by taking the public key hash, XOR'ing it with the descriptive string (everything following the slash in the URI) and hashing the result once again [6, p. 50].

This means, that the key for the stored data is derived from only a version string and the public key of the publisher of the information.

### 4.3.2. The Meta Table

We propose using something similar to SSKs in IgorFs to store and look up the IgorFs address of our encrypted (hash, key) data. We create a new hash table that only contains this information. This table will be called "Meta Table" from now on. We chose this name, because in its pure form this table is only used to store the information we need to find our data.

We use an algorithm to calculate the hash keys for the Meta Table entrys that is similar to the one that is used to calculate the SSKs in Freenet: We take the hash of the public key $p$ and the hash of the number of the file system revision $r$. We concatenate these two hash values and hash once again. The result is the Meta Table key $l$ we can use to lookup the information about one specific revision of the file system.

$$l = H(H(p) + H(r))$$

We put the information we need to lookup the encrypted data block that is stored in IgorFs at the address of this key. We use this indirect method of storing the encrypted data, because the amount of encrypted data can (potentially) be large (several megabytes of encrypted data for one file system revision are possible) and because of the way we distribute our updates. Furthermore, data that is inserted into IgorFs is initially only stored on the the node who inserts the data and only propagated within the distributed file system when another node requests it. The same should hold true with our file system revision data.

Every client can predict this value, the only thing it needs to know is the public key of the publisher. In principle, it also needs a file system version number to start with. But if if that is impossible the client simply may begin counting at "1".

A sample data lookup is presented in Figure 4.3 on the following page: We have the public key of the publisher, a version number and the key data we got from the publisher. Now we hash both the public key and the version number independently of each other. After that we concatenate the results of these operations and hash the concatenated value once again. Now we perform a DHT lookup of this value. The result is a signed IgorFs (ID, key)-tuple. We verify the signature with the publishers public key. Now we use IgorFs to retrieve the data block that is stored at the (ID, key)-tuple. The result is a signed data block that is encrypted with the subset difference scheme we presented above. We verify the signature against the public key of the publisher and try to decrypt it. If the decryption

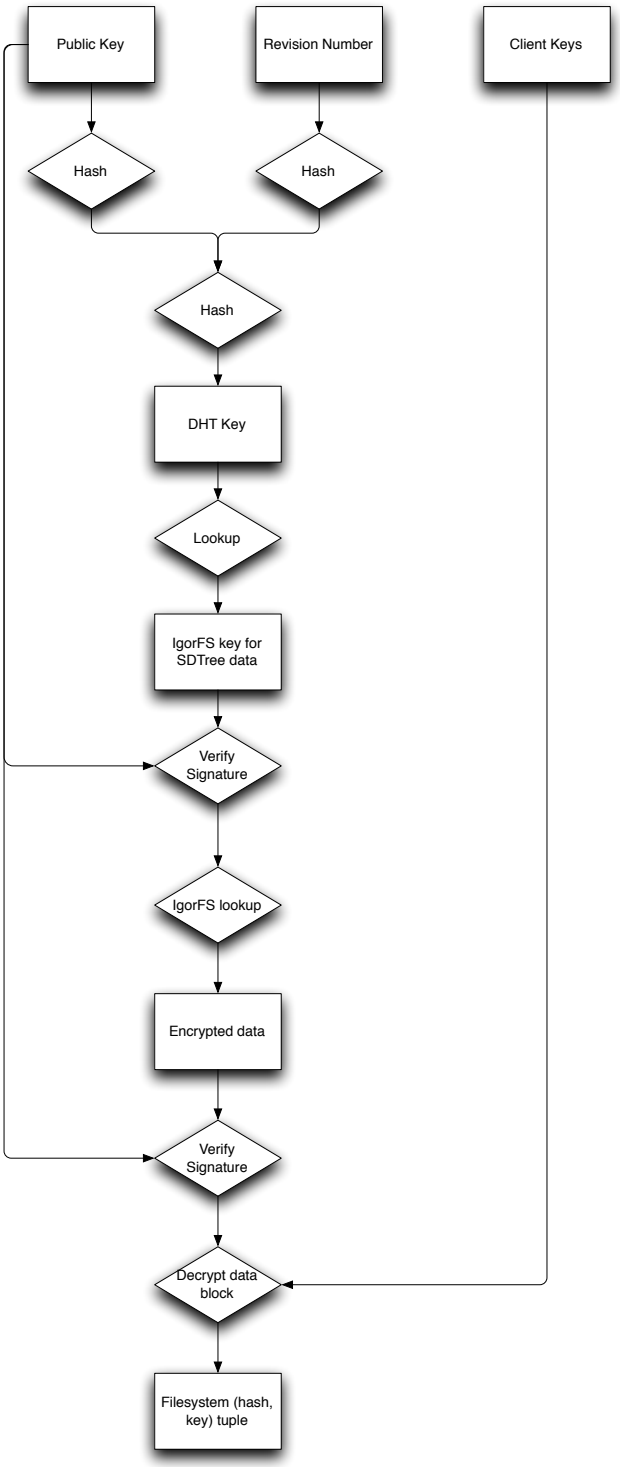



Figure 4.3: Sample lookup

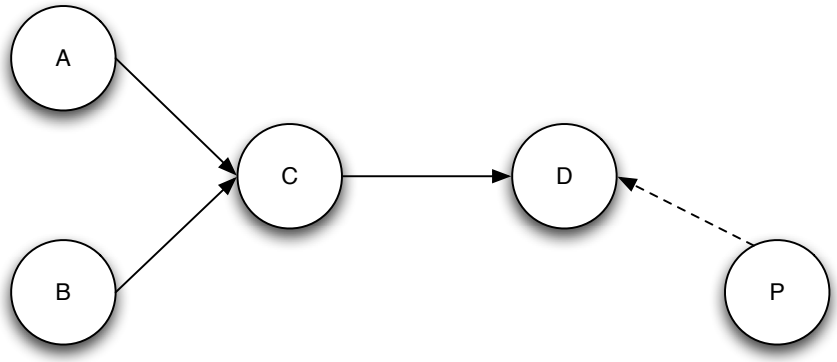



Figure 4.4: Soft-state multicast request

is successful the result is the (ID, key)-tuple of the root entry of the IgorFs file system.

When a publisher wants to insert a new (hash, key)-tuple of the file system root, it simply encrypts it with our encryption scheme, signs the result and stores the data block in IgorFs. After that the publishing node takes the resulting IgorFs (hash, key)-tuple, adds some more Information (like the file system revision and the publisher key), signs this data and inserts it into the Meta Table. The Meta Table only accepts entrys that have a valid cryptographic signature (to prevent the insertion of garbage or damaged data).

### 4.3.3. Soft-State Multicast

The pull approach has one big disadvantage to the multicast approach: it is slower. A new file system revision is not pushed to the clients, it is only retrieved when the client tries to get a new revision. It would be desirable to speed this up a little bit and our approach enables us to do this with very little overhead.

To realize this, we simply let the clients keep track of the Meta Table-requests that were forwarded by it.

We will illustrate this idea with the help of Figure 4.4. This picture shows a network with 4 nodes that are important for us. $A$ and $B$ are clients that have mounted a certain IgorFs file system. They are both at revision 1 and are trying to request details about revision 2. $D$ is the node that is responsible for the Meta Table entry of revision 2. So both $A$ and $B$ send their requests for revision 2 to $D$. The requests are routed via $C$ and $C$ remembers that $A$ and $B$ requested revision 2. After being forwarded to $D$, the requests arrived at their destination. Filesystem revision 2 is not yet available, so node $D$ replies with a "file system revision not found" and notes that it received a request for file system revision 2 that was sent by $C$. He does not note $A$ and $B$ individually, it only notes the last hop that forwarded the request.

Now $P$ inserts file system revision 2 and sends the entry to $D$. Node $D$ stores the information about the new file system revision and remembers that someone already requested this file system revision. It then forwards the information to $C$, which in turn forwards the information to $A$ and $B$.

In principle we once again create a multicast tree via reverse-path forwarding and forward our new file system revision via this new multicast tree. To make the performance even better, the publisher $P$ could send the revision to the node that was responsible for revision 1 and this node could then try to send the message to all nodes that requested revision 1 and are still in the list.

We call this technique soft-state multicast, because it is only an addition to our pull-mechanism and as long as a node that requested a certain revision is still in the lists it will get the new revision automatically as soon as it is published. After a certain amount of time it will drop out of the lists. But the node will try to get the new revision every few seconds or minutes (depending on the polling interval that was chosen), so it will appear on the lists again. And even when it doesn't get a new revision via the multicast but only via the pull-mechanism it will just take a little longer.

### 4.3.4. Efficient Revision Storage

It can already be seen from the lengthy explanations required that the subset difference scheme is rather complicated and the overhead introduced by it is quite massive. The message size of the encrypted blocks also can get rather big (several hundred kilobytes or even megabytes of data if many users have been revoked in the tree). In addition to that we need to create a whole new encrypted block for every new file system revision with our current strategy. But is this really necessary?

If we look at the scheme, we use the subset difference revocation scheme only to encrypt the key, that we use to encrypt our data. But this key does not have to be changed for every file system revision. As long as no users are revoked, we could simply use the same key to encrypt the (ID, key)-tuples instead of creating a whole new subset difference encrypted block.

The overhead would get considerably smaller with this approach, we only have to store the tuple, that amounts to approximately 160 bits for the ID and a maximum of 256 bits for the key - that are 53 bytes of data compared to up to a few megabytes of data for the subset difference encrypted blocks.

Because of this, we introduce the concept of "major" and "minor" file system revisions. Every time, a user is revoked, the major file system revision is incremented and a new subset-difference block with a new master-key is created. All subsequent file system revisions only increment the minor revision number and use the master-key from the major revision. For minor revisions now also store the encrypted data directly in the Meta Table - the amount of data is small enough to not require the usual layer of indirection.

Furthermore, when a new major revision is started because of a revocation, we insert an "end-marker" at the last revision. If e.g. the actual major revision is 5 and the minor revision is 230, we insert the end-marker as minor revision 231 and then start major revision 6. When a client reads an end-marker, it knows that a new major revision has been started.

# 5. Cryptographic Implementation

Now that we covered the basics, we will examine the implementation that solves our problem in greater detail. This section will explain our specific implementation of the cryptographic functions used later on in this project. We will only explain the details that were newly developed. Knowledge of the working of common cryptographic algorithms like AES [48] or hashing algorithms like SHA [49] is a prerequisite.

Like already explained earlier in this paper we use the Subset Difference Revocation Scheme to encrypt our data. This enables us to encrypt our data in one big data block, that everyone who has got the right keys may decrypt, and allows the revocation of every single person that once had a valid set of decryption keys.

As the encryption scheme was already explained in 4 on page 25 we will not explain it here again. For our usage scenario we decided to use a tree with a height of $h = 32$. That means that we potentially can have $2^{32} - 1$ or 4294967295 clients. We do not have $2^{32}$ clients because at least one node always has to be revoked for the encryption scheme to work.

## 5.1. Key Generation

The first step we must take to be able to use our encryption scheme is the key generation. We have got a tree with a height of 32. Every node of this tree needs a unique, non-predictable secret key. We call these keys "master keys". Only the publisher knows the master keys of the entire tree, the clients only know keys that are derived from these master keys.

The best way to get the master keys would be a true random-number source. The keys would be unique and non-predictable. The problem with this approach is, that the publisher would have to remember the keys. It would not need to create all $2^{32}$ keys at once, in the beginning it only needs a small fraction of keys (exactly $h$ if it has got one client, 32 in our case) and it could generate new keys on-demand when they are needed. But our system should be scalable and this system would get slower with every new client that is added to it. The whole tree has got $2^{33}$ nodes. If our master key is 128 bits in length, this would require 8 gigabytes of data. Furthermore a non-random system lets us easily delegate subsets of the tree to other authorities (see section 5.2 on page 38).

Because of that we decided to generate the keys by using a hash algorithm. The data that is hashed consists of the path $p$ of the specific node in the tree (32 bits in our case),

the level $l$ of the node in the tree and a secret string $s$, which only the publisher knows. These three pieces of data combined form a unique sequence of bits for each node in the tree. After hashing it with a strong hash algorithm we should get a unique unpredictable key for each node. When the secret string is of sufficient length it should not be possible to calculate the values of other keys in the tree, no matter how many keys one already does know.

We use SHA-256 [49] as a hash algorithm. Because we need only 128 bits of data, but SHA-256 returns 256 bits of data, we XOR the first and the second 128 bits and use the result. If SHA-256 should be broken at some later time it is quite trivial to replace the algorithm that is used with another one, that is more secure.

The master key for each node is hence calculated as follows ( $\mathrm{cut}_{a,b}(x)$ being a function that returns bits $a$ to $b$ of argument $x$):

$$k = \mathrm{cut}_{0,127}(\mathrm{SHA256}(p + l + s)) \oplus \mathrm{cut}_{128,255}(\mathrm{SHA256}(p + l + s))$$

A possible improvement would be, to use the Blum-Blum-Shub generator [3] to generate our master keys. This random number generator is not very fast, but it produces very strong pseudo-random number. There exists a proof which ties the complexity of predicting random numbers output by this generator to the complexity of integer factorisation.

The next step is the algorithm for the generation of or left, right and centre keys for each subtree. For the generation of this we use AES in a mode similar to the AES counter mode (for a explanation of the CTR mode see e.g. [22, 37]).

We use this mode of operation, because AES is a fast encryption algorithm and it is possible, that we have to execute this operation quite a few times (especially when we are encrypting data and the revoked users are scattered around the tree). Furthermore AES should be resilient to known-plaintext attacks. We exploit this fact in the following way:

We need to generate three 128 bit output blocks from one 128 bit input blocks. It must not be possible to guess the input block from any or all the output blocks. If we use AES to encrypt incrementing numbers and use our input block as key, we can generate as many different output blocks, as we need. We know the plaintext to each of these blocks but we cannot deduce the encryption key that was used from the plaintext or from the cryptotext. This approach satisfies our stated requirements.

That means that we encrypt the value “1” for left, “2” for centre and “3” for right with the appropriate key. The result of this operation is the key for the left, centre or right node respectively.

The functions for the generation of the keys are therefore defined as:

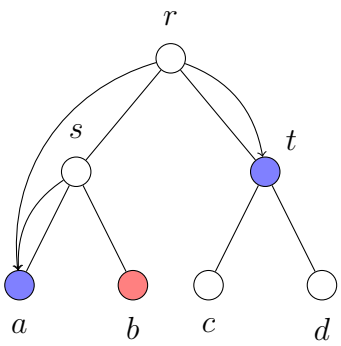



Figure 5.1: Key distribution for $h = 2$

$$G_L(x) = \mathrm{AES}_{128}(1, x);$$
$$G_C(x) = \mathrm{AES}_{128}(2, x);$$
$$G_R(x) = \mathrm{AES}_{128}(3, x);$$

Now we have got a method for creating our master keys and calculating all the subsidiary keys in the subtrees. The next step is establishing a way to find out all the keys that we have to give to each client.

Each client has to be able to calculate all keys of all subtrees it is a part of, except the keys that are on a direct path to the root node. A little example is shown in Figure 5.1: We want to generate all the keys for the node $b$ (coloured in red) with the path $p = 01$ (0 meaning left and 1 meaning right). The node $b$ naturally is a member of the tree rooted at $r$. Furthermore the node $b$ is a member of the subtree rooted at $s$, but it is not a member of the subtree rooted at $t$.

The keys we give to $b$ are the keys of the node $t$ and $a$ in the tree rooted at $r$ and the key of $a$ in the tree rooted in $s$. One can easily verify, that with this information, $b$ can determine all the keys in the trees rooted at $s$ and $r$, except the keys that are on a path to the root from itself.

An algorithm that generates these keys is easily devised. We walk down the tree from the root to our target node. After each step we add the key of the other child of the parent of the actual node to the key list, and repeat this step for every possible parent of the actual node.

In our example, we start at the root, $r$. The first node on the way to $b$ is $s$. Now we add the key of the other child of the parent node $s$ to the key list (with the root node $r$). After that we walk our next step and arrive at $b$. The other child of the parent node $s$ is $a$. The two possible root nodes are $r$ and $s$. Consequently we add the key of $a$ to the key list – one time for the tree rooted at $r$ and one time for the subtree rooted at $s$. The final

| Path | Subset Depth | Difference Depth | Key |
|---|---|---|---|
| 10 | 0 | 1 | $G_R(r)$ |
| 00 | 0 | 2 | $G_L(G_L(r))$ |
| 00 | 1 | 2 | $G_L(s)$ |

Table 5.1: Example key list for Figure 5.1

key list is presented in Table 5.1.

The number of keys that have to be distributed to a participant in a tree with a height of $h$ and $N$ participants can be calculates with $\frac{h(h+1)}{2} + 1$ or $\frac{1}{2}\log^2 N + \frac{1}{2}\log N + 1$. In our case where $h$ is 32 we can support up to $2^{32} - 1 = 4294967295$ users and every user has got 529 keys. With a key length of 128 bit, a path-length of 32 bit and the subset and difference depth each being saved as a 32 bit integer (64 more bits) that amounts to 14812 bytes of key data for the algorithm that each client has to save. The actual data file for clients is a little bit larger and varies for each client. An test client used had a key file size of 14841 bytes. The exact structure of the client key file is shown in Table 5.3 on page 42.

## 5.2. Distributed Key Generation

The following idea was not included in our enhancements to IgorFs, but the implementation would be rather straightforward if the usage scenario requires it.

It would be nice if we were able to delegate the task of key distribution to other institutions. Lets say we are a publisher called $M$ and have got a file system with medical data and a new institution $I$ with 10000 computers shall get access to the data. We could generate all the keys ourselves or we could delegate the key generation and distribution process to the institution.

With our method of key generation this approach is pretty straightforward. We simply use different secrets for different parts of the tree and tell these secrets to the authorities we delegate the key generation to. Lets look at Figure 5.1 on the previous page again and assume that we want to delegate key generation for every node below $s$ to a third party. We choose a secret for the whole tree e.g. “oursecret”. After that we choose a secret for the subtree starting at $s$, e.g. “‘secondsecret”. We tell $I$ the path of $s$, the secret “‘secondsecret” and $G_R(r)$.

$I$ now can calculate the keys of all nodes in their subtree and has got the additional keys it cannot create itself. To add a new user $I$ simply generates the keys of its subtree, adds $G_R(r)$ to the key list and distributes these keys to the new client. If a client in this subtree is to be revoked this information has to be conveyed back to the publisher $M$. $I$ cannot revoke users itself.

$M$ can calculate all the trees simply by looking which secret word is associated with a

| Path | Secret |
|---|---|
| 000/0 | oursecret |
| 000/1 | secondsecret |

Table 5.2: Table of secrets for a small tree

specific subtree. This could be realized with a longest-prefix-matching algorithm. If $M$ wants to know the secret for a specific node it simply looks at the most specific entry in the table. The table for our example is shown in Table 5.2. The most specific entry can be found by doing a longest-prefix-match that is very common for IP addresses. Longest prefix matching is a well-researched topic and there are several fast algorithms available. For a comparison of approaches to longest-prefix matching please see e.g. [60].

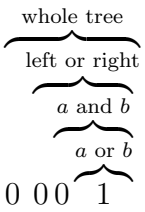
## 5.3. User Revocation

For our revocation scheme to work we need at least one revoked user. So now we will first address the matter how we handle the revoked users. The list of revoked users is saved by the server. But we not only can revoke single users, we are also able to revoke whole subtrees of users and we have to be able to save this data efficiently. It would be stupid if we had to save e.g. 16384 revoked users if we want to revoke a subtree of height 16.

Usually the paths that identify a specific user are saved as 32-bit values, each bit representing one right/left decision in the tree. We could simply save the depth of the node we want to revoke and ignore the rest of the path. But we decided to use another way that simplifies the algorithms that are used later. The decision was to double the length of the value that is used to save the path of the nodes (from 32 to 64 bits in our case) and store the depth of the revoked node there. This approach greatly simplifies our work later on.

For an example let us look at the tree in Figure 5.1 again. We want to revoke the subtree $s$ that contains the users $a$ and $b$. User $a$ has the path 00, user $b$ has got the path 01. The expanded path of user $a$ is 0000, the expanded path of user $b$ is 0001. The path that revokes both $a$ and $b$ is 0010. 0110 would cover $c$ and $d$, 1000 theoretically would cover the entire tree.

The following illustration shows again, what each bit represents (the example is again for Figure 5.1). The highest bit represents that the whole tree is covered. The next bit represents the choice if we walk left (to $s$) or right (to $t$) in the tree. The next bit represents the choice of the nodes below $s$ are covered and the last bit represents the choice of $a$ or $b$ (because we walked to $s$ in the second step).




The nodes that are revoked are now simply stored as a (sorted) list. When a new node is revoked, we have to check if the new node is covering any other nodes, that are already revoked, and if this is the case remove them from the list. E.g. when 0001 is already revoked and we revoke 0010 (meaning 0001 and 0000) 0001 has to be removed from the list of revoked node.

We also have to test, if a node that we revoke is already revoked, because a revoked node in a higher layer covers this part of the tree. In this case, we may not add this new node to the list of revoked nodes.

The entire file format the server uses to store the information about revoked users is shown in Table 5.4 on page 43.

## 5.4. Cover Generation

Lets assume that we have got a message $M$ that we want to encrypt and a list of revoked users $\mathcal{R}$ who shall not be able to decrypt our message. We already know how the encryption works in principle: we chose our nodes in a way, that no revoked node can calculate the keys and encrypt the master key with these keys.

We identify these nodes by their path and their subset and difference height. To illustrate the meaning of subset and difference height we once again use Figure 5.1. We want to tell the clients to use the ID of node $A$ as a key, in the subtree that is rooted at $i_2$. In this case, the path would be 0000, the subset depth would be 1 (meaning that the subtree that we have chosen begins at layer 1) and the difference depth would be 2 (meaning that the node we picked is rooted at layer 2).

We will collect all these nodes in a list, use the node keys to encrypt the master keys and send all this information together with the encrypted message to the clients. Each of the authorised clients will be able to deduce the right keys from his key informations, all the revoked clients (individually or together) cannot deduce the keys from their key data.

The next step now is to create an algorithm that can calculate, which nodes we have to use, that no revoked users but every un-revoked user can deduce the keys. An algorithm for this is shown in [42, p. 11] and reproduced here to explain the way in which we implemented it:

The method given in [42] partitions $\mathcal{N}\backslash\mathcal{R}$ into disjoint subsets $S_{i_1,j_1}$, $S_{i_2,j_2}$,..., $S_{i_m,j_m}$ in the following way:

Let $ST(\mathcal{R})$ be the (directed) Steiner Tree induced by $\mathcal{R}$ and the root. The subsets collection is built iteratively, maintaining a tree $T$ which is a subtree of $ST(R)$ with the property, that any $u \in \mathcal{N}$
$\mathcal{R}$ that is below a leave of $T$ has been covered. We start by making $T$ equal to $ST(\mathcal{R}$ and then iteratively remove nodes from $T$ (while adding subsets to the collection) until T consists of just a single node:

1. Find two leaves $v_i$ and $v_j$ in $T$ such that the least-common-ancestor $v$ of $v_i$ and $v_j$ does not contain any other leave of $T$ in its subtree. Let $v_l$ and $v_k$ be the two children of $v$ such that $v_i$ is a descendant of $v_l$ and $v_j$ is a descendant of $v_l$. If there is only one leave left, make $v_i = v_j$ to the leave, $v$ to be the root of $t$ and $v_l = v_k = v$.
2. If $v_l \not\equiv v_i$ then add the subset $S_{l,i}$ to the collection. Likewise, if $v_k \not\equiv v_j$ add the subset $S_{k,j}$ to the collection
3. Remove from $T$ all the descendants of $v$ and make it a leave.

The interesting part about this algorithm is our specific implementation of it. We represent the revoked receivers $R$ by a list of 64-bit bit values. These values are stored in a sorted list. We implement the tree steps of the algorithm like this:

1. The first step of our algorithm is finding two leaves such that the least-common-ancestor does not contain any other leave of $T$ in its subtree. We implement this by looking at the first three values in our sorted list. Let $p_1, p_2, \ldots, p_n$ be the path of the first, second, ..., $n$th node in our list. We now define $p_c = \neg p_1 \wedge p_2$ and $p_d = \neg p_2 \wedge p_3$. If $p_c \leq p_d$ the second set of nodes with the paths $p_2$ and $p_3$ cover a bigger part of the tree than the first set of nodes. That means that the least-common-ancestor $v$ of $p_1$ and $p_2$ does not contain any other leave of $T$ in its subtree and we can continue with our algorithm. If $p_c > p_d$ the second pair covers a smaller part of the tree than the first one and we restart our algorithm again starting at $p_2$ until we found a fitting pair.

   This calculation works, because we are using a sorted list. That means, that $p_1$ is always smaller than $p_2$. When we calculate $p_c$, we negate $p_1$ and use the binary "or" of the result with $p_2$. The resulting number has got the highest-bit 1 set at the place, that the paths of $p_1$ and $p_2$ began to divergate. That means, that we have to cover this part of the tree with our algorithm.
2. We now calculate the layer $r = l(p_c << 1)$ of the least-common-ancestor by calculating

$$r = l(p_c << 1) = (p_c << 1) \wedge 0xAAAAAAAAAAAAAAAA$$

   and searching the first bit that is set to one starting at the most-significant bit. $r$ is the offset of the first bit set to 1 we found from the highest-order bit divided by two. We calculate the layers of $p_1$ and $p_2$ by calculating $l(p_1)$ and $l(p_2)$.

   We already explained, that we moved from 32 to 64 bit paths, because it makes certain things easier later on. This is exactly the place, where we have got an advantage by

| Data | Length |
|---|---|
| Publisher Public Key Length | 32 Bits |
| Publisher Public Key | Dynamic |
| Client Path | 32 Bits |
| Path 1 | 32 Bits |
| Subset Depth 1 | 32 Bits |
| Difference Depth 1 | 32 Bits |
| Key 1 | 128 Bits |
| Path 2 | 32 Bits |
| ... | ... |
| Path 529 | 32 Bits |
| Subset Depth 529 | 32 Bits |
| Difference Depth 529 | 32 Bits |
| Key 529 | 32 Bits |

Table 5.3: Client Key File

using these long trees. We can now simply calculate the layer of $p_c$ by left-shifting it by 1. This works, because $p_c$ has got the highest order 1 bit set at the place, where the paths of $p_1$ and $p_2$ diverge, as already explained above. If we left-shift this number by one, we shift the 1 bit from the "path" bits to the "cover" bits, that show that this region of the tree is to be covered. This is exactly what we want to know.

If the difference of $r$ and $l(p_1)$ is greater than one we add the path of $p_1$ with the subset depth $r+1$ and the difference depth $l(p_1)$ to our list $\mathcal{S}$ that contains all subset-difference entries. If the difference of $r$ and $l(p_2)$ is greater than one, we do the same thing with $p_2$.

3. After that we remove $p_1$ and $p_2$ from $T$ and add $p_1 \vee (p_c << 1)$ to $T$.

## 5.5. Message Signature

We sign several of our messages to be able to ensure that only messages, that originate from the publisher are accepted by the clients. The clients can verify the authenticity of a message by verifying its signature. We use ECC (Elliptic Curve Cryptography) signatures for these operations. We use ECC signatures, because they have approximately the same security as traditional signature algorithms like RSA- or ElGamal, but have significantly shorter key lengths. For example to achieve the same level of security as 1024-bit RSA with elliptic curves requires a key size of about 171–180 bits [63, p. 4]. Furthermore the complexity of the operations is much lower that for traditional algorithms [13, p. 340].

Many public-key cryptosystems today are based on the discrete logarithm problem

| Data | Length |
|---|---|
| Publisher Private Key Length | 32 Bits |
| Publisher Private Key | Dynamic |
| Tree Secret Length | 32 Bits |
| Tree Secret | Dynamic |
| Filesystem Revision | 32 Bits |
| Revoke list length | 32 Bits |
| Revoked Path 1 | 64 Bits |
| ... | ... |
| Revoked Path n | 64 Bits |

Table 5.4: Server File

(DLP) in a group [64, p. 134]. The DLP can be shortly defined like this: Let $G$ be a group and $\circ$ its group operation. Let $x$ and $y$ be elements of $G$ and $n \in \mathbb{N}$. Define $y = x \circ x \circ \cdots \circ x$ with $n$ group operations or $y = x^n$. The DLP consists of determining the value of $n$ only knowing $x$ and $y$ and the group properties of $G$. Examples for DLP public-key cryptosystems are ElGamal [17] or DSA [45][13, p. 378].

ECC was first proposed in the mid-eighties by Victor Miller[41] and Neil Koblitz[31] independently. Elliptic curve cryptography is based on the fact that a group operation can be embedded into the solution of an elliptic curve. In this way one can obtain a group that can be used to do cryptographic calculations. Most interestingly all DLP-based cryptosystems can be converted to use elliptic curves [64, p.136].

In our case we chose to use the Curve P-384 [46, p. 32] which has got a key length of 384 bits and was specified by the NIST for use by US government entities. We use the OpenSSL implementation[20] of ECDSA (Elliptic Curve Digital Signature Algorithm) [67].

To sign our message we first hash the content with SHA-1 [11, 49] and then use ECDSA to sign this hash. E.g. in Table 5.5 on page 46 everything from the magic number to the encrypted message is hashed and cannot be manipulated later on.

## 5.6. Message Encryption

Now that we have got our cover we only have to encrypt our message. This step is fairly straightforward: we have to choose one random 128 bit master secret and another random 128 bit initialisation vector (IV), which has to be unpredictable. It particularly has to be impossible to predict the IV that will be associated to the plaintext in advance of the generation of the IV [see 47, p. 20].

We use `/dev/random` as a random source. In Linux this is a “real” random number generator, that uses entropy generated by e.g. interrupts because of network or hard disc

activity or CPU clock timings. There are some considerations about the security of this generator. To be more exact there was among other problems, some discussion about the possibility of a break in forward-security in [21]. But the considerations in that paper mostly are only relevant to the urandom pool [21, sec. 3.1], which we do only use for the generation of nonces. The urandom pool also never was meant to provide real high-quality random data. The forward security considerations also applies only to the urandom pool. The primary random pool blocks when there is not enough entropy to generate more random numbers and should be secure enough for our application.

We encrypt our data with 128 Bit AES in CBC-Mode [47, p. 10]. The AES key is generated by reading 128 Bits of data from the random pool of the operating system. The initialisation vector, which is needed for AES in CBC mode is generated by reading 128 Bits from the urandom pool and then applying the encryption operation with our AES key to the read random data. This approach is recommended in [47, App. C]. We do not use the random pool for the generation of IVs for the reason, that we don't want to exhaust the random pool that the operating systems offers. This pool usually does not contain very much data (about 1000 Bit seems to be a relatively common number after a longer run-time). A new IV is needed for every single encryption operation - and the single requirement that an IV has to satisfy is, that it has to be non-predictable. We also could use a counter and encrypt it which the AES-key every time (this is another recommended way of [47, App. C]) - but then we would have to keep track of the counter.

After the generation of our AES master key, we encrypt it with each of the keys we got by using the Subset Difference method. Now every client that can generate one of the keys created by the Subset Difference method can decrypt our AES master key. The message that contains all the information we need for the decryption is shown in Table 5.5 on page 46.

All the steps that are needed to encrypt a message are once again shown in Figure 5.2 on the following page. First, we take the list of revoked users and generate our cover and look up all the node IDs that are later used to encrypt the AES master key. The second step is to generate this master key and encrypt it with each of the generated cover keys. In the third step, we take our message and encrypt the message with the AES master key. After that we use SHA-1 to hash the information that was generated up till now. We use our elliptic curve key to sign this hash and append the signature to our output.

## 5.7. Message Decryption

Now we will use this short section to explain, how a encrypted message is decrypted again.

The first step in message decryption is to parse the message, generate the SHA-1 hash of the message contents and check the validity of the signature. If the message signature is not valid, we return at once and do not try to continue the decryption process. If the signature is valid, we have to search the different paths with which the master key is encrypted and

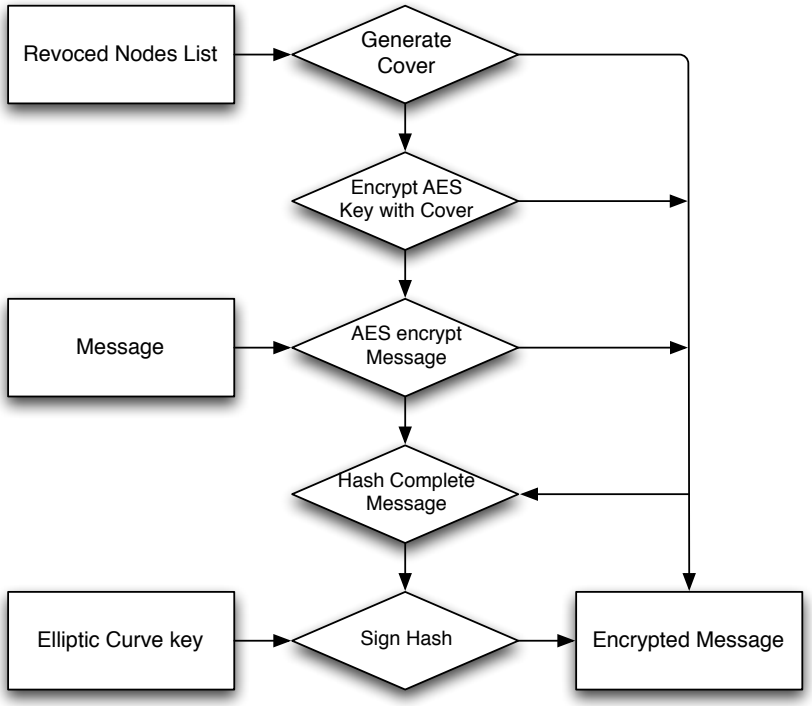



Figure 5.2: Message Encryption

| Data | Length |
|---|---|
| Magic Number (77235) | 32 Bits |
| Public Key Length | 32 Bits |
| Public Key | Variable |
| SD Keylist Size | 32 Bits |
| Path 1 | 32 Bits |
| Subset Depth 1 | 32 Bits |
| Difference Depth 1 | 32 Bits |
| Path 2 | 32 Bits |
| ... | ... |
| Difference Depth $n$ | 32 Bits |
| Encrypted Masterkey 1 | 128 Bits |
| Encrypted Masterkey 2 | 128 Bits |
| ... | ... |
| Encrypted Masterkey $n$ | 128 Bits |
| Encrypted Message Size | 32 Bits |
| AES IV | 128 Bits |
| Encrypted Message | Variable |
| Signature Size | 32 Bits |
| Signature | Variable |

Table 5.5: Encrypted Message Container Format

| Data | Length |
|---|---|
| Magic Number (723623) | 32 Bits |
| Block ID | 256 Bits |
| Crypto Key | 256 Bits |
| Data Flag | 8 Bits |
| Endmarker Flag | 8 Bits |
| Data | Variable |

Table 5.6: Encrypted Message

try to determine if we can use one of them to decrypt the master key.

The process for doing this is rather straightforward: for every path in the encrypted message we look in our key store and look if we can use one of the paths in our key store to generate the right key for the message path. When we successfully found a path we can decrypt, we determine the AES master key and use this key to decrypt the message.

If we don't find a path that we can use to decrypt the master key, that means that we have been revoked. In this case we simply abort. We cannot decrypt the message.

# 6. Implementation in IgorFs

The IGOR File System is built in a modular manner. The program consists of several modules which are "glued" together by a messaging system which is used by the different modules to communicate with each other. Each module can send messages to the any other module.

## 6.1. IgorFs Modules

First we will give a short overview of the modules with the highest importance for this work with IgorFs. These modules are:

- The IGOR interface. This module is responsible for the message exchange with IGOR. Every message that is sent via the network has to go through this module. Messages that are received from the network are forwarded to the responsible module.
- The FUSE interface. This module is responsible for the communication to the virtual file system layer of the operating system through the fuse library. All file system operations that are triggered by the operating systems are handled by this module and converted in requests that are handled by the other modules.
- The Pointer Store. This module is responsible to manage the information which block IDs are present on which node of the network. In principle this is the indirection layer that maps the DHT entries of blocks to their real position on the different nodes.
- The Block Cache, block transfer and block fetcher modules handle the caching and transfer of blocks over the network.
- The File and Folder handling module. This module connects the fuse interface with the block transfer module.
- The Meta Table. This module was newly implemented and stores the information about available file system revisions. It works similar to the pointer store and will be described more in-depth later in this chapter.
- The Snapshot Initiator. This module creates new snapshots of the root file system in predefined time intervals and inserts the appropriate data in the Meta Table. The module only runs on nodes that publish a file system.
- The Snapshot Requestor. This module only runs on nodes, that have are subscribers of a file system. It tries to get new file system revisions from the Meta Table

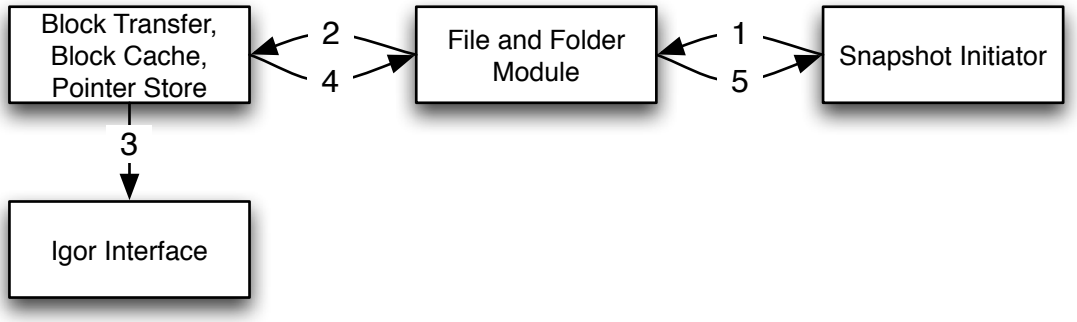



Figure 6.1: Classic snapshotting

- The Subset Difference module. This module implements all the routines required for the subset difference revocation method as described in 5.

## 6.2. Snapshotting in IgorFs

First we will explain how snapshotting was implemented in IgorFs before the changes and extensions of our paper. In IgorFs each file system has a superblock. This superblock usually is only stored in memory. The snapshot initiator periodically sends a snapshot timer message to the file and folder module. This is shown in Figure 6.1 in step number one. The file and folder handling module makes all necessary files persistent (that means that the block-cutting, etc. is started and also the new directory entries are inserted into the DHT). After that the superblock is encapsulated into a block and sent to the block transfer module (step two). The block transfer module performs some more communication which is not really interesting for us and in the end the data is sent via IgorFs (step 3). The block transfer module sends the block ID and crypto key of the superblock back to the file and folder module (step 4). This module forwards the information back to the snapshot initiator (step 5) which in turn writes the information to the log file.

The exportkey command can be used to extract this information from the logfile. After that an additional IgorFs node which uses the saved superblock can be started.

## 6.3. Snapshotting with the Meta Table

Now we explain how the new snapshotting mechanism works in detail and which new features were implemented.

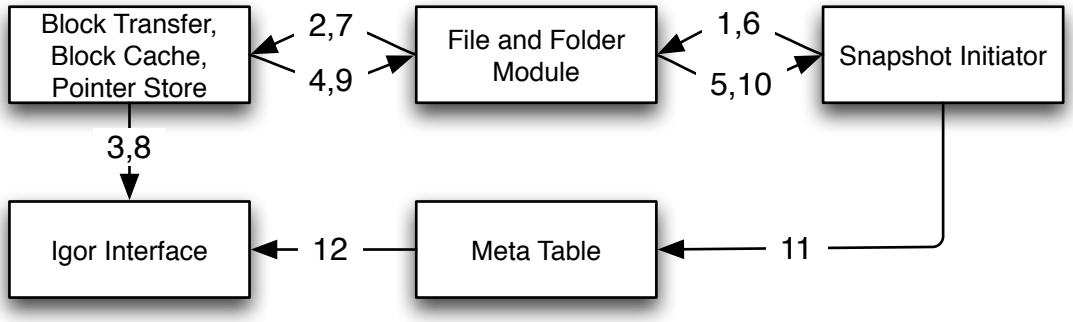



Figure 6.2: Sample snapshot insertion

### 6.3.1. Creating Snapshots

The first few steps are exactly the same as they were without the Meta Table. In step one we send a snapshot request to the file and folder handling module. The module makes the files persistent and sends the new superblock to the block transfer module which in turn forwards it to other modules and in the end a message is sent via IGOR (steps two and tree). The block transfer module sends the answer to the file and folder module which forwards the block ID and crypto key to the snapshot initiator (steps four and five).

Up till now that was not different to the old way things were handled. But now the shapshot initiator encapsulates the block ID and hash key and encrypts this encapsulated message using the subset difference algorithm. The actual encryption is done by the subset difference module with a single function call. The encrypted message is now again sent to the file and folder module with a request to insert the block into IgorFs (step 6). The file and folder module forwards the block once again to the block transfer module and forwarded further (step 7,8). The file and folder module gets the block ID and crypto key of this new inserted block (step 9) and forwards it to the snapshot initiator (step 10). The snapshot initiator sends the information about the block ID and crypto key to the Meta Table (step 11) which in turn propagates it to IgorFs (step 12).

The big difference to the way things were handled before is, that we now encrypt the pieces of information, that we need to access the file system and store them in file system itself. After that we create a pointer to this information and store it in an additional hash table. Everyone can access this table and retrieve the encrypted pieces of information that we stored beforehand. But only the people in possession of valid crypto keys are able to decrypt the pieces of information and access the file system.

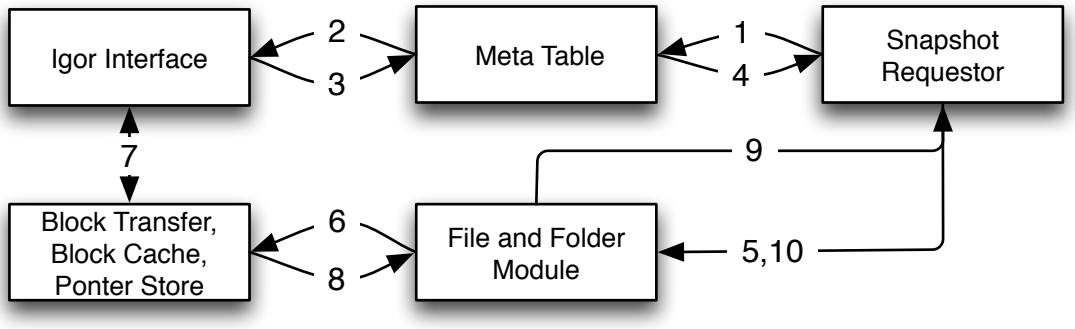



Figure 6.3: Sample request

### 6.3.2. Using Snapshots

What a client does to retrieve the information is pretty straightforward and shown in Figure 6.3. In the first step, the client requests an specific snapshot from the Meta Table. The Meta Table looks if it can satisfy this request on its own - if it cannot it forwards the request to other IgorFs nodes (steps 2 and 3). The result of the operation is sent back to the snapshot requestor (step 4). The result can either be a "not found", if no file system revision that matches the request has been found in the Meta Table or the pointer to the encrypted file system information. If we received the encrypted file system information, we send a request to the File and Folder Module and quickly receive the encrypted block (steps 5 to 9). After that we can decrypt the block and replace the superblock of the file system (step 10).

After that we regularly ask the Meta Table (in the current implementation every 20 seconds), if a new version of our file system is available.

### 6.3.3. Enhancing the Efficiency

It was already explained in section 4.3.4 on page 33, that at the moment, every time our file system is changed we invoke our subset difference scheme and create the message shown in Table 5.5 on page 46. We realize at first glance, that this block is quite complicated and big. Furthermore the generation and decoding is not trivial.

The solution to this problem was already shown: we use the same AES key that was used to encrypt the subset difference message for every following file system revision and only generate a new AES key, when a user is revoked. This new shorter container format for out message is shown in Table 6.1 on the next page.

| Data | Length |
|---|---|
| Magic Number (77237) | 32 Bit |
| Message Size | 32 Bit |
| AES IV | 128 Bit |
| Encrypted Message | Variable |

Table 6.1: Short Encrypted Message Container Format

## 6.4. The Network Messages

In this section the network messages that are sent and received by the Meta Table are examined and explained.

Basically, there are three different messages that are sent via the IGOR network. These messages are:

- Insertion Request
  This message is sent, when a new file system revision is inserted into the Meta Table. The message contains file system major and minor revision, the (id, key)-tuple for the subset difference block that has been stored in IgorFs and (when the minor revision is greater than zero) the AES-encrypted (hash, key)-tuple for the newest file system revision. Further more the whole message contains the EC public key of the publisher and is signed by the publisher.
- Content Request
  This message is sent, when a node tries to get a file system revision from the Meta Table. It contains the public key of the file system publisher, the major and minor revision as well as a flag, if we want to get our specific revision or if any file system revision that is newer than the requested revision is acceptable. The message also contains the identifier of the requesting node.
- Content Reply
  The content reply is sent by the node that is responsible for the requested file system revision. There are two different possible types of answer.

  The first answer is a "file system revision was not found" that is sent, if fitting file system revision has been found in the Meta Table.

  The answer contains once again the file system minor and major revision (the returned revision could be newer than the requested one), the (id, key)-tuple for the subset difference tree block, (if present) the AES-encrypted (hash, key)-tuple for the newest file system revision as well as the publishers EC public key and signature.

The Meta Table identifiers for file system revisions, that are identical to the destination addresses for each of the request messages, are calculated by hashing the EC public key of the publisher as well as the minor and major revisions with SHA-256.

### 6.4.1. On the fly Request Filling

When a client requests a new file system revision, it sends the content request message to the appropriate IgorFs node. While the message is travelling to its final destination, every intermittent node also looks at the message. If one of these intermittent nodes is able to fill our request, the intermittent node answers the request and sends it back to the source.

But why should an intermittent node be able to fill our request, if only the responsible node is storing the Meta Table entries?

We take the same approach, that was taken with the Pointer Store and already explained in section 2.3.4. When a node requests a Meta Table entry and received a valid reply, it stores the new entry in his own Meta Table. Now when a request to this revisions comes by this node, it is able to fill the request. This approach could potentially lead to serious speed-increases in certain situations.

Suppose, that we are publishing a file system, that is subscribed by a large institution that uses several hundred or thousands of computers to access our file system. All these systems reside in one network, our publishing system resides in another network that has only got a high-latency connection to the institutions network. Now several of the institutions node request our Meta Table entry and store it in their own Meta Table. Because we are using Proximity Route Selection and Proximity Neighbour Selection it is probable that after some time a sizeable portion of the requests will only be handled in the institutions network.

### 6.4.2. Requesting new File System Revisions

At the moment, requesting new file system revisions is somewhat problematic. When a new client joins the file system, it would have to ask for each and every file system revision, until it arrives at the current revision. If only one revision somehow "got lost" the node can never get to the newest revision, because it will keep re-requesting the hole thinking that its current revision is the newest available.

Fixing the hole issue is simple. The client just requests the next four or five revisions instead of just the next one. (Freenet takes the same approach). This gives us a speed-up of looking up new revisions... but it still is very slow. File system revisions could go into the tens of thousands or even more and it would still take a rather large amount of time to find the newest revision.

A second related problem is the problem of data loss or timeout. The Meta Table cannot store an arbitrary amount of data, at some time it needs to start deleting old data that probably is of no big importance because everyone already is using newer revisions. Similarly at the moment data is written only once to the Meta Table. Ff a node that saved some Meta Table entries leaves the network or if the responsibilities for the ID-space change, some data may no longer be accessible.

The solution to this problem is to re-insert certain Meta Table entries into the table in

regular intervals. Interestingly this problem has a connection to the problem of how we request the newest file system revision. We will elaborate on this in greater detail later, but let us keep it in mind for the remainder of this section.

One solution we propose quickly access find the newest file system revision is, to use an exponential search algorithm, that is only used until the newest file system revision is found. The algorithm starts, by asking for the file system revision $2^0 = 0$. If this revision is found, it asks for $2^1 = 2$, if that is found $2^2 = 4$ and so on, until the first negative reply is received. Let's assume, that we have tested up to $2^9$ and received a "not found" back from the Meta Table. After that, we will still check for $2^{10}$ and $2^9 \pm 1$, just for the case that the entry for $2^9$ was lost. If all these requests turn up negative replys, we jump back and try the file system revision try the revision $2^9 - 2^7 = 2^8 + 2^7$. If this revision is found, we jump to $2^8 + 2^7 + 2^6$, if it is not found, we jump to $2^8 + 2^7 - 2^6$. We continue in this way until we found the newest available revision. We should make every step a little bit fuzzy and always probe not only one specific version, but three versions lying next to each other, in case a Meta Table entry got lost.

In general, we want to find the newest revision $x$. For this, we try to access revision $2^i$ beginning with $i = 0$. When we find the first revision that is not available and the last good revision was $2^j$, we try accessing $2^j + 2^{j-1}$. After that if we find the revision we continue by adding $2^{j-2}$, if we didn't find it, we substract $2^{j-2}$. We stop once we reach $2^0$ and have thus found the newest available revision.

As already explained, we chose to use two revision numbers for our file system: the major and the minor revisions. But our searching scheme can easily be adjusted to this change: we first search for the largest major revision and start the search for the minor revision afterwards.

If we choose this searching algorithms, we can exploit it to minimise the space requirements of the Meta Table: old entries time out of the Meta Table after a certain amount of time. To prevent data loss, we have to re-insert our data from time to time. The trivial solution would be for a publisher to remember all and every Meta Table entries and to republish them regularly. As we already can see this is quite pointless and not very efficient. If we know, that our clients use the exponential request scheme explained above to search for the newest revisions, we only have to insert the revisions which a client will request while searching for the newest revision.

Re-insertion of the Meta Table entries is a very simple task. The publisher keeps track of all the revisions it already sent out and periodically resends the important revisions. The interval of these re-sendings should not be too short. Meta Table entries are not very memory consuming, the length of an entry should be approximately 150 bytes (it varies a bit because the signature length is not constant). That means that we can save about 7000 Meta Table entries in one megabyte of memory. Because of that, we can choose to store a rather large amount of these entries without too much concern about our memory – and the entries should not time out too quickly. Even if a new entry is added every second, and we only store 7000 entries, the first timeout would occur after about 2 hours. And it

is highly unlikely, that a new entry is added in this kind of speed.

While this approach has many advantages, especially in speed and memory requirements, it also has problems. With this approach, every time a client joins the network, the same hosts are queried for the new version – and it would be optimal, if the load would always be distributed on different hosts. It is the opinion of this author, that this probably would lead to no performance or load problems, unless a massive amount of hosts tries to join at the same time, but the problem persists nevertheless.

Another problem should be considered, especially in large networks with small probing intervals for the newest revision: the load on the node, which will host the newest file system revision can get relatively high, if it has to answer to all requests for a new file system revision and only sends back a "not found". Even abandoning the "not found" reply and just discarding the packets would only cut the network traffic in halve. Caching negative results in the network for a specified amount of time would be one posible solution, but in IgorFs this idea poses problems. Because of the IGOR routing algorithm packets will nearly never take the same path from destination to source than they took from source to destination. Packets that are sent back to the destination by our overloaded node will even after the first hop arrive at a node whose ID is relatively similar to the source of the request. But this node will very likely never see any request packet and it does not help at all, if it caches a negative reply. PRS and PNS change this a little bit, but the overall problem remains. It would be possible to send the reply packets back the same path the request took, but this opens up a whole bunch of other problems

A second possible solution would be to store Meta Table entries on several hosts instead of just a single one. This could be requested by an overburdened network node. When a node notices, that it becomes overburdened with requests, it adds a note to the "not found" messages, containing the information, that the load is becoming too high and if the new Meta Table entry arrives, it will be mirrored on $x$ other nodes. The hosts that got this reply can now choose a random number $y \leq x$ for each request and calculate the ID of the mirror nodes by hashing the usual information and appending this new mirror identifier to it. (When they choose $y = 0$ they don't append the mirror identifier at all, but ask the original target). When the new revision is inserted, it will be automaticially distributed to all these mirror hosts. This should not amplify the insertion of new revisions very much but it can take a significant load from single nodes of the network.

Due to time-constraints the ideas presented in this section were not itegrated into our program. Our implementation only asks for the next $x$ minor/major revisions and does not use an exponential scheme.

# 7. Testing IgorFs

This section describes, how IgorFs with our new subsystem was tested for its functionality.

Under optimal circumstances, a system like IgorFs should be tested in a scenario that is as similar to the real world as possible. A big number of hosts on the Internet that are distributed around the world would be a good start.

Usually PlanetLab is used for tests like this. PlanetLab has been founded in 2003 and is a research network which as of November 2007 consists of 825 nodes at 406 sites. Affiliated institutions can use PlanetLab to deploy and test their applications on a global scale.

Unfortunately IgorFs (in contrast to e.g. IGOR) can not be tested with PlanetLab. IgorFs is dependent on the FUSE library which is not present (and cannot be compiled) on PlanetLab.

Because of this, we had to resort to simpler methods and all tests of IgorFs were only conducted in local area networks. This means, that the data produced by the test results is not entirely reliable. When deployed over the Internet, reaction times will be much slower due to the network latency times.

At the moment also no testing framework for IgorFs exists; all test instances of IGOR and IgorFs have to been started manually.

Another problem that had to be taken into account while testing IgorFs was the dependence on IGOR as the overlay network. At the time this work was written, a major revamp of IGOR was in progress. Needless to say, this had an impact on the overall stability of the available IGOR versions; there were, among others, routing problems.

IgorFs also was under ongoing development during the time span of this thesis; the IgorFs version that was used in the tests has many known issues, but most of the issues have no adverse effect on our new modules.

Because of this, most of the tests were performed on a rather old IGOR version[1]. Unfortunately this old version is missing some of the more attractive new features like Proximity Route Selection and Proximity Neighbour selection.

[1] more exactly: IGOR version 0.2.3.

## 7.1. Testing Environment

To test IgorFs with our new instances, a little script which creates the client keys and can start an arbitrary number of IGOR and IgorFs instances on a host was written. With the help of this script IgorFs instances were then started on several different machines within a network.

The interesting thing about the clients is, which file system revision they are currently using. This is always the last revision they were able to (successfully) get from the network and should be equal to the latest file system revision.

To be able to easily test, which file system revision the clients are using, a new entry to the IgorFs proc file system was added. The IgorFs proc file system works in the same manner, the proc file system of Linux and many Unix distributions work. The proc file system contains virtual files, which can be used to access information or statistics about the system; a few of them can even be used to change system parameters – one can change certain kernel settings like e.g. enable or disable IP forwarding by writing data to them.

The IgorFs proc file system is available in "`/proc`" at the root of the IGOR File System. Like the operating systems proc file system it contains several virtual files, that can be accessed and provide information about the file system, like e.g. the current (hash, ID)-tuple of the superblock or now also the current major and minor revision the client is using.

The test was then committed, by using several computers within a local network and starting a number of IgorFs instances (four in our case) on each one of them. After that, we used the publisher to insert new file system revisions into the Meta Table.

In our tests, the time after which each client tries to get a new version was set very low (five seconds). The time after which the server inserts a new file system revision if there were changes also was set quite low (ten seconds). This allowed us, to quickly insert many file system revisions into the Meta Table. In the real world, one would probably use a much larger timescale for these timers - a sensible example would be one test if there is a new file system revision available per five minutes.

But all this heavily depends on the way IgorFs is to be used; one even could think of applications where snapshots are only created on request and not because of an timer.

## 7.2. Test Results

The tests did not reveal any surprising results. Inserting and retrieving file system revisions worked without problems. Due to the very low latency of the network, all clients managed to track new revisions in a very small time span; it took mere seconds after a new file system revision was inserted into the Meta Table until it appeared on all the subscribed nodes. The largest amount of time until a client got the newest file system revision was the five seconds until a client probed whether the new files system revision was present.

Because of this, no exact measurements of the time a lookup takes were made; this simply makes no sense in a network of this size.

Due to the relatively small size of our network it also took very few tries for a client that connected to an already existing file system to get the newest revision. To simulate the behaviour of IgorFs in a slightly larger network, we disabled the caching of Meta Table entries.

This test yielded no unexpected results. A new client that joined the file system had to probe several times, until it reached the new file system revision, and because of this, the whole lookup procedure took longer. But the lookup, encryption and decryption worked as expected and no unforseen problems were encountered during the time of the tests.

# 8. Conclusion and Outlook

In this work, we extended the IGOF File System to provide an secure, fast and asynchronous update notification mechanism for the mounted file systems. Several possible detailed solutions were shown until we settled on the Subset Difference revocation algorithm.

The Subset Difference revocation scheme allows us to encrypt data in a way that enables us to send messages that only authorised clients can decrypt. We can add new clients or revoke old ones and prevent them from decrypting any new messages. The clients are stateless.

This cryptographic algorithm is then used to encrypt the update information and store it in the DHT at a predetermined address which changes for every file system revision. It can be retrieved from there by all subscribers of the file system. Subscribers continually poll the DHT for new file system revisions.

The approach we use has several similarities to the way updates are handled in Freenet, but also has some important differences. The first notable difference is, that Freenet itself implements no structured Peer-to-Peer network. Instead it uses its keys for a flooding search approach. Furthermore, Freenet only uses a public-key system to encrypt its data. This system only prevents that unauthorised sources can create new revisions of files. There is no client authorisation of any kind. There is only one decryption key – if this key is "in the wild" anyone can decrypt any revision of the file. There is even no possibility to a new revision in a way that allows us to revoke users. (To be fair this never was the intention of Freenet. It only wanted to provide anonymity and does a good job at that).

Moreover, our technique has several similarities to the way multicast key exchanges are handled. In our final implementation we use the Subset Difference method only to encrypt an AES key, which is used in all further updates, unless a change in the set of revoked users occurs. In this case a new AES key is generated and once again encrypted with the Subset Difference method. This second operation would be the equivalent of a re-keying in a multicast network.

Our approach also has several limitations that have to be considered. The approach we chose usually provides no backwards-confidentiality. One could ostensibly start with a fully revoked tree and only allow users one by one. But this either creates a very large amount of Subset Difference data, once the number of users grows or limits the way in which we can assign the keys.

Forward-confidentiality is granted though: a user that is revoked can not read any new

file system revision. He can still access the old files with the old (hash, ID)-tuples of the supernodes of the IGOR File System he was subscribed to. This is a technical restriction and simply cannot be helped.

A further limitation that has to be considered is the resistance of IGOR and IgorFs to denial of service attacks. The approach we presented in this thesis allows us to securely send update notification to authorised subscribers. A malicious user that has no keys should not be able to access our file system at all. He can however very easily interrupt the service for other users. IGOR is based on the idea that every node forwards data for other nodes. If a node decides to break this rule or to send corrupt data to other IGOR nodes, it can with relative ease create a widespread service disruption.

During the development of our final approach, we found many ideas that still could be added to the IGOR File System. A few of them have already been addressed in this work; for example the soft-state multicast that was explained in section 4.3.3 on page 32 could be added to our implementation. Furthermore, when the need arises, the distributed key generation explained in section 5.2 on page 38 could be implemented without bug changes to the code base.

But there are also several other points, that should be subject to scrutiny: At the moment, all the data in the Meta Table is only inserted once, and it is assumed that it won't drop out of the table. In a fluctuating network, this assumption is most certainly wrong. Nodes will drop out and all the Meta Table entrys will be lost. Nearly the same problem arises, when a new system joins the Network. The responsibility of nodes for the hash-range near the ID of the new node will change. The new node will be responsible for certain entries of the Meta Table, but it won't have any knowledge of these entries because they never were sent to it. A solution to this problem would be a nice addition, several ideas were already shown in section 6.4.2.

Another nice addition would be the use of the Blum-Blum-Shub pseudo number generator, because of his proved security (see section 5.1).

I conclude this thesis with the words of an Irish philosopher and hope that we succeeded in statisfying this maxim.

*Wisdom denotes the pursuing of the best ends by the best means.*

Francis Hutcheson [24]

# A. Tool Usage

This appendix will give a short overview about the usage of the command-line tools that are required to use IgorFs.

## A.1. File System Administration Utility

The "File System ADMinistration utility" (fsadm) was newly implemented and is used to handle and generate filesystem revisions. With this utility one can create the public and private keys for a new filesystem, generate the key files for clients and revoke users.

We will first explain the options for this program and then give a short step for step guide on how to create a new filesystem from scratch.

### A.1.1. Usage

When run with the option "-h" the utility outputs the following command line options:

```
Help:

 h: display this help
 s: server key file
 o: client key output file
 v: verbose
 k: key to generate client file for
 r: key to revoke
 d: depth at which to revoke the key
 w: write server file
 e: set tree secret
 g: generate cover
```

The options "h" and "v" are self-explanatory, the other options are explained in more detail below.

- Write
  When the option **w**rite (w) is specified, the elliptic curve private key, filesystem

revision and revoke list is written to a file named "serverkey". If no such file is present, it is created. The structure of this file is shown in Table 5.4 on page 43.
This option should be specified, when a new filesystem is created for the first time and every time a user is revoked.

- Revoke
  When the option **r**evoke (r) is specified, the path to a node that is to be revoked is expected as a second argument. The node to be revoked is specified by a path consisting of ones and zeroes, beginning at the bottom layer of the tree. Additionally the major filesystem revision is incremented.
  The "r" option should always be used in conjecture with "w", otherwise the changes are not saved.
  Example:
  `./fsadm -wr 10000000000000000000000000000000`
- Client key generation
  When the option "k" is specified, the path to a node is expected as a second argument. Fsadm will create the keys that the specified node needs to access a filesystem. The format for the path of the node is exactly the same as for the option "r".
  This option should always be used in conjecture with "o", otherwise nothing is written to disk. When the option "v" is specified, the generated keys will be shown on the screen.
- Client key output file
  When the option "o" is specified, the path to an output file is expected as a second argument. Fsadm will output the generated client keys to this file.
  The option "k" also has to be specified, otherwise no action is taken.
  Example:
  `./fsadm -vk 10100010101110000100000000000000 -o cfile`
- Revoke depth
  With the option "d", it is specify to revoke whole subtrees at once. Starting the layer that is specified as a second option, every node in the path is revoked
  This option has to be specified with "-r", otherwise it has got no effect.
  Example:
  `./fsadm -wr 00000000000000000000000000000001 -d 1`
  This example would revoke every node whose path begins with 1 (that is half of the nodes in whole the tree.)
- Server key file
  The option "s" allows you to change the name of the server key file. The name defaults to "serverkey".
- Generate cover
  The option **g**enerate cover (g) is mostly a debug function that allows you to encrypt an arbitrary string you have to supply as an second argument. The encrypted data is simply output to the screen, you have to redirect the output to a file to be able to

do something useful with this option

- Set tree secret
  With the option "e", you can set the secret that is used to generate the IDs for the nodes of the master tree. This should be a secret word of sufficient length; anyone who knows this secret is able to decrypt everything.

### A.1.2. Creating a new File System

To create a new filesystem from scratch, we have to generate a new server key an revoke a node. The encryption scheme always needs at least one revoked node to work. Furthermore we have to set our tree secret to something suitable. We can do this all in one step or at the same time, it does not matter:

```
./fsadm -we "This is a long secret but not very secure"
./fsadm -wr 10000000000000000000000000000001
```

Now we can generate the keys for one client:

```
./fsadm -o cfile -k 10000000000000000000000000000000
```

After that we just have to copy the file to our client node and can startup the different IgorFs instances.

## A.2. The Client Utility

The client utility is a little example program, that shows how to use the subset difference module for decryption. The utility is called "client". It reads the client key data from a file named "cfile" and tries to decrypt a file named "rev". The encrypted file can e.g. be created by using `fsadm` with option -g.

The source for the client utility can be found at /src/sdtree/client.cc starting from the IgorFs base folder.

## A.3. IgorFs

In this paper, we have enhanced the functionality of IgorFs. But by default our new distribution method is disabled - without any options IgorFs behaves exactly the same as before our new additions

### A.3.1. Usage

We added the following new command-line-switches to IgorFs:

- Use new snapshotting
  With the option "-i", out new snapshotting functions are enabled. An IgorFs instance started with -i assumes that it shall publish its own filesystem. It will try to load the server key data from a file named "serverkey" residing in the path it was started from. After that it will begin inserting new filesystem revisions into the meta table.
- Use new snapshotting client
  With the option "-c", we enable the client functions for our new snapshotting algorithms. IgorFs will try to read the client key data from a file called "cfile", residing in the path that IgorFs was started from. After that it will try to get from the metatable starting at major revision 1, minor revision 1.
  This option implies "-i".

### A.3.2. Starting our new File System

Starting the new filesystem is not very difficult. We already explained in section A.1.2 on the previous page how to create the keys that are needed now.

For the server we assume that the file "serverkey" already has been created and that an IGOR instance has been started and is listening at localhost port 6666. The file system shall be mounted at /igorpublish. IgorFs then can be started by issuing the following commands:

```
export IGOR=localhost:6666
./igorfs -i /igorpublish
```

For the client, we assume that the file "cfile" has been created and that an IGOR instance is listening at localhost port 9999. The file system shall be mounted at /igorsubsc. IgorFs can be started by issuing:

```
export IGOR=localhost9999
./iorfs -c /igorsubsc
```

# B. Source Configuration Options

In several of the source files, there are a few switches, that can be changed to adjust the behaviour of certain components of IgorFs. A short list and explanations of all these options will be given here.

Options in `src/sdtree/fpublish.cc`

- DEBUGOUTPUT
  This option is used in several source files and enables additional very verbose debug output routines to the screen that are of no use to normal users, but could be of help when there are problems. The most important file, where this option is also present is fclient.cc. A similar option exists in signature.cc.

  Like already mentioned the files in the sdtree subdirectory do not use the usual IgorFs logging routines, because they can compile independently from IgorFs.
- RANDOMFILE
  This option is used to set the source for random numbers that are used for the generation of the AES keys. Usually `/dev/random` is used here.
- IVRANDOMFILE
  This option is used to set the source for random numbers that are used for the generation of the AES initialisation vector. Because security is not really important, only uniqueness is required usually `/dev/urandom` is used here.

Options in `src/sdtree/sdtcommon.h`

- aes_bits
  This option sets the number of bits that is used for the AES key. It usually is set to 128 bits; when changing this option, further changes to the source are required; e.g. the generation functions for the keys still will output only 128 bits.
- tree_height
  This option sets the height of the tree that is used for the subset difference algorithm. Please note that after changing this option further changes in the source may be required.

Options in `src/sdtree/signature.hh`

- ECCURVENAME
  This option sets the elliptic curve that is used for our signatures. This usually is set to NID_secp384r1, but can be set to any curve that is supported by OpenSSL.

Options in `src/metatable/metatable.hh`

- mMaxMetaTableEntrys
  Thos option sets the maximal number of metatable entries, that are stored on this node.

Options in `src/snapshot/SnapShotInitiator.cc`

- SNAPSHOTTIMER
  Number of seconds after which a new revision is created and inserted into the metatable (if there have been changes to the file system).
- REVISIONFILENAME
  File name of the proc file which contains the information about the current revision

Options in `src/snapshot/SnapShotRequestor.cc`

- REQUESTORTIMER
  Number of seconds, after which the snapshot requestor tries to get a new revision of the filesystem from the metatable.
- REVISIONFILENAME
  File name of the proc file which contains the information about the current revision the client is using.

# List of Figures

# List of Tables

# Bibliography


[1] AACS LA. *Advanded Access Content System (AACS): Introduction and Common Cryptographic Elements*. Revision 0.91, February 2006. URL `http://www.aacsla.com/specifications/specs091/AACS_Spec_Common_0.91.pdf`.

[2] BBC iPlayer, accessed 1st November 2007. URL `http://www.bbc.co.uk/iplayer/support/`.

[3] L. Blum; M. Blum and M. Shub. A Simple Unpredictable Pseudo-Random Number Generator. *SIAM Journal on Computing*, Volume 15 (2), May 1986.

[4] M. Castrom; P. Druschel; A.-M. Kermarrec and A. I. Rowstron. Scribe: A Large-Scale and Decentralized Application-Level Multicast Infrastructure. *IEEE Journal on Selected Areas in Communications*, Volume 20 (8), October 2002.

[5] I. Clarke; S. G. Miller; T. W. Hong; O. Sandberg and B. Wiley. Protecting Free Expression Online with Freenet. *IEEE Internet Computing*, Volume 6 (1): pp. 40–49, 2002.

[6] I. Clarke; O. Sandberg; B. Wiley and T. w. Hong. Freenet: A Distributed Anonymous Information Storage and Retrieval System. In *Designing Privacy Enhancing Technologies: Proceedings of the International Workshop on Design Issues in Anonymity and Unobservability*, Volume 2009 of *LNCS*, pp. 46–64. Springer Berlin / Heidelberg, 2001.

[7] R. Clayton. The rising tide: DDoS from defective designs and defaults. In *SRUTI'06: Proceedings of the 2nd conference on Steps to Reducing Unwanted Traffic on the Internet*, pp. 1–6. USENIX Association, Berkeley, CA, USA, 2006.

[8] B. Cohen. Incentives Build Robustness in BitTorrent. May 2003. URL `http://www.bittorrent.org/bittorrentecon.pdf`.

[9] Y. K. Dalal and R. M. Metcalfe. Reverse Path Forwarding of Broadcast Packets. *Communications of the ACM*, Volume 21 (12): pp. 1040–1048, 1978.

[10] DFN-PKI, accessed 1st November 2007. URL `https://www.pki.dfn.de/`.

[11] D. Eastlake and P. Jones. US Secure Hash Algorithm 1 (SHA1). RFC 3174, Internet Engineering Task Force, September 2001.

[12] J. Eberspächer and R. Schollmeier. First and Second Generation of Peer-to-Peer Systems. In R. Steinmetz and K. Wehrle (editors), *Peer-to-Peer Systems and Applications*, Volume 3485 of *LNCS*, pp. 35–56. Springer Berlin / Heidelberg, 2005.

[13] C. Eckert. *IT-Sicherheit*. Oldenbourg, München, 4th edition, 2006.

[14] eDonkey2000, accessed 1st November 2007. URL `http://edonkey2000.com/`.

[15] J. Eickhold. Entwicklung einer SSR-basierten Peer-to-Peer-Telefonieanwendung für das Nokia 770. Master's thesis, System Architecture Group, University of Karlsruhe, Germany, 2006.

[16] The Freenet Project: Understand Freenet. accessed 1st November 2007. URL `http://freenetproject.org/understand.html`.

[17] T. E. Gamal. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. In *Advances in Cryptology: Proceedings of CRYPTO 84*, Volume 196. Springer Berlin / Heidelberg, 1985.

[18] The Annotated Gnutella Protocol Specification v0.4. Revision 1.6, accessed 1st November 2007. URL `http://rfc-gnutella.sourceforge.net/developer/stable/index.html`.

[19] K. Gummadi; R. Gummadi; S. Gribble; S. Ratnasamy; S. Shenker and I. Stoica. The Impact of DHT Routing Geometry on Resilience and Proximity. In *SIGCOMM '03: Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, pp. 381–394. ACM, New York, NY, USA, 2003.

[20] V. Gupta; D. Stebila and S. C. Shantz. Integrating Elliptic Curve Cryptography into the Web's Security Infrastructure. In *WWW Alt. '04: Proceedings of the 13th International World Wide Web Conference on Alternate Track Papers & Posters*, pp. 402–403. ACM Press, New York, NY, USA, 2004.

[21] Z. Gutterman; B. Pinkas and T. Reinman. Analysis of the Linux Random Number Generator. *2006 IEEE Symposium on Security and Privacy*, May 2006.

[22] R. Housley. Using Advanced Encryption Standard (AES) Counter Mode With IPsec Encapsulating Security Payload (ESP). RFC 3686, Internet Engineering Task Force, January 2004.

[23] R. Housley; W. Polk; W. Ford and D. Solo. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 3280, Internet Engineering Task Force, April 2002.

[24] F. Hutcheson. *Inquiry into the Original of our Ideas of Beauty and Virtue*. 1725. Treatise I, sec. v, § 18.

[25] M. Izal; G. Urvoy-Keller; E. W. Biersack; P. Felber; A. A. Hamra and L. Garcés-Erice. Dissecting BitTorrent: Five Months in a Torrent's Lifetime. In *Passive and Active Network Measurement*, Volume 3015 of *LNCS*, pp. 1–11. Springer Berlin / Heidelberg, 2004.

[26] Joost, accessed 1st November 2007. URL `http://joost.com/`.

[27] D. Karger; E. Lehman; T. Leighton; R. Panigrahy; M. Levine and D. Lewin. Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. In *STOC '97: Proceedings of the twenty-ninth Annual ACM Symposium on Theory of Computing*, pp. 654–663. ACM Press, New York, NY, USA, 1997.

[28] KaZaA, accessed 1st November 2007. URL `http://kazaa.com/`.

[29] Y. P. Kising. Proximity Neighbor Selection and Proximity Route Selection for the Overlay-Network IGOR. Diplomarbeit, Lehrstuhl für Netzwerkarchitekturen, Technische Universität München, 2007.

[30] T. Klingberg and R. Manfredi. Gnutella 0.6. RFC Draft, June 2002. URL `http://rfc-gnutella.sourceforge.net/src/rfc-0_6-draft.html`.

[31] N. Koblitz. Elliptic Curve Cryptosystems. *Mathematics of Computation*, Volume 48 (177): pp. 203–209, January 1987.

[32] K. Kutzner; C. Cramer and T. Fuhrmann. A Self-Organizing Job Scheduling Algorithm for a Distributed VDR. In *Workshop "Peer-to-Peer-Systeme und -Anwendungen", 14. Fachtagung Kommunikation in Verteilten Systemen (KiVS 2005)*. Kaiserslautern, Germany, February 2005.

[33] K. Kutzner and T. Fuhrmann. The IGOR File System for Efficient Data Distribution in the GRID. In *Proceedings of the Cracow Grid Workshop CGW 2006*. Cracow, Poland, 2006.

[34] R. M. Lerner. At the Forge: Syndication with RSS. *Linux Journal*, Volume 2004 (126): p. 8, 2004.

[35] E. Levy and A. Silberschatz. Distributed File Systems: Concepts and Examples. *ACM Computing Surveys*, Volume 22 (4): pp. 321–374, 1990.

[36] D. Lewin. Consistent Hashing and Random Trees: Algorithms for Caching in Distributed Networks. Master's thesis, Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, 1998.

[37] H. Lipmaa; P. Rogaway and D. Wagner. Comments to NIST Concerning AES-modes of Operations: CTR-mode Encryption. In *Symmetric Key Block Cipher Modes of Operation Workshop*. Baltimore, Maryland, USA, 2000.

[38] J. Lotspiech; M. Naor and D. Naor. Subset-Difference based Key Management for Secure Multicast. IRTF Draft, Internet Research Task Force, July 2001.

[39] C. Lowth. Securing your Network against Kazaa. *Linux Journal*, Volume 2003 (114): p. 3, 2003.

[40] P. Maymounkov and D. Mazières. Kademlia: A Peer-to-Peer Information System Based on the XOR Metric. In *Peer-to-Peer Systems: First InternationalWorkshop, IPTPS 2002 Cambridge, MA, USA, March 7-8, 2002. Revised Papers*, Volume 2429 of *LNCS*, pp. 53–65. Springer Berlin / Heidelberg, 2002.

[41] V. S. Miller. Use of Elliptic Curves in Cryptography. In *Advances in Cryptology - CRYPTO '85: Proceedings*, Volume 218. Springer Berlin / Heidelberg, 1986.

[42] D. Naor; M. Naor and J. Lotspiech. Revocation and Tracing Schemes for Stateless Receivers. In *Advances in Cryptology - CRYPTO 2001: 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, Volume 2139 of *LNCS*. Springer Berlin / Heidelberg, 2001.

[43] R. M. Needham and M. D. Schroeder. Using Encryption for Authentication in Large Networks of Computers. *Communigations of the ACM*, Volume 21 (12): pp. 993–999, 1978.

[44] R. M. Needham and M. D. Schroeder. Authentication revisited. *SIGOPS Operating Systems Review*, Volume 21 (1): pp. 7–7, 1987.

[45] NIST. The Digital Signature Standard proposed by NIST. *Communications of the ACM*, Volume 35 (7): pp. 36–40, 1992.

[46] NIST. Digital Signature Standard (DSS). Federal Information Processing Standards Publication 186–2, U.S. Department of Commerce/National Institute of Standards and Technology, January 2000.

[47] NIST. Recommendation for Block Cipher Modes of Operation – Methods and Techniques. NIST Special Publication 800–38A, U.S. Department of Commerce/National Institute of Standards and Technology, December 2001.

[48] NIST. Specification for the Advanced Encryption Standard (AES). Federal Information Processing Standards Publication 197, U.S. Department of Commerce/National Institute of Standards and Technology, November 2001.

[49] NIST. Specifications for the Secure Hash Standard. Federal Information Processing Standards Publication 180–2, U.S. Department of Commerce/National Institute of Standards and Technology, August 2002.

[50] M. Nottingham and R. Sayre. The Atom Syndication Format. RFC 4287, Internet Engineering Task Force, December 2005.

[51] J. Postel. Transmission Control Protocol. RFC 793, Internet Engineering Task Force, September 1981.

[52] A. Rowstron and P. Druschel. Pastry: Scalable, Decentralized Object Location and Routing for Large-Scale Peer-to-Peer Systems. In *IFIP/ACM International Conference on Distributed Systems Platforms (Middleware)*, pp. 329–350. November 2001.

[53] O. Sandberg. Distributed Routing in Small-World Networks. In *8th Workshop on Algorithm Engineering and Experiments (ALENEX06), Miami, FL, USA*. January 2006.

[54] Skype, accesed 1st November 2007. URL `http://skype.com/`.

[55] Steam, accessed 1st November 2007. URL `http://steampowered.com/`.

[56] I. Stoica; R. Morris; D. Karger; M. F. Kaashoek and H. Balakrishnan. Chord: A scalable Peer-to-Peer Lookup Service for Internet Applications. In *SIGCOMM '01: Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, pp. 149–160. ACM Press, New York, NY, USA, 2001.

[57] D. Tam; R. Azimi and H.-A. Jacobsen. Building Content-Based Publish/Subscribe Systems with Distributed Hash Tables. In *Databases, Information Systems, and Peer-to-Peer Computing: Proceedings of the First International Workshop, DBISP2P 2003 Berlin, Germany, September 7 - 8, 2003 Revised Papers*, Volume 2944 of *LNCS*, pp. 138–152. Springer Berlin / Heidelberg, 2004.

[58] Tu. Legitime Interessen. *Frankfurter Allgemeine Zeitung*, (67): p. T 1, 3rd March 2007.

[59] I. Voras and M. Zagar. Network Distributed File System in User Space. In *28th International Conference on Information Technology Interfaces*, pp. 669–674. Cavtat/Dubrovnik, 2006.

[60] M. Waldvogel; G. Varghese; J. Turner and B. Plattner. Scalable High-Speed Prefix Matching. *ACM Transactions on Compututer Systems*, Volume 19 (4): pp. 440–482, 2001.

[61] D. Wallner; E. Harder and R. Agee. Key Management for Multicast: Issues and Architectures. RFC 2627, Internet Engineering Task Force, June 1999.

[62] K. Wehrle; S. Götz and S. Rieche. Distributed Hash Tables. In R. Steinmetz and K. Wehrle (editors), *Peer-to-Peer Systems and Applications*, Volume 3485 of *LNCS*, pp. 79–93. Springer Berlin / Heidelberg, 2005.

[63] M. J. Wiener. Performance Comparison of Public-Key Cryptosystems. *CryptoBytes*, Volume 4 (1): pp. 1–5, 1998.

[64] E. D. Win and B. Preneel. Elliptic Curve Public-Key Cryptosystems - An Introduction. In *State of the Art in Applied Cryptography: Course on Computer Security and Industrial Cryptography, Leuven, Belgium, June 1997. Revised Lectures*, Volume 1528 of *LNCS*. Springer Berlin / Heidelberg, 1998.

[65] J. Winkelhage. Die große Verstopfung. *Frankfurter Allgemeine Zeitung*, (244): p. 22, 20th October 2007.

[66] C. K. Wong; M. Gouda and S. S. Lam. Secure group communications using key graphs. In *SIGCOMM '98: Proceedings of the ACM SIGCOMM '98 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, pp. 68–79. ACM Press, New York, NY, USA, 1998.

[67] Public Key Cryptography for the Financial Services Industry, The Elliptic Curve Digital Signature Algorithm (ECDSA). Technical report, Accredited Standards Commitee X9, Inc., 2005.

[68] Extensible Markup Language (XML) 1.0 (Fourth Edition). August 2006. URL `http://www.w3.org/TR/REC-xml/`.

[69] Zattoo, accessed 1st November 2007. URL `http://zattoo.com/`.